

Lecture Topics

- Introduction: What is Computer Graphics?
- Raster Images (2D image representation, manipulation and compositing) Assignment 1.
- Ray Casting (camera, visibility, normals, lighting, Phong illumination)
- Ray Tracing (shadows, supersampling, global illumination)
- Spatial Data Structures (AABB trees, OBB, bounding spheres, octree)
- Meshes (connectivity, smooth interpolation, uv-textures, subdivision, Laplacian smoothing)
- 2D/3D Transformations (Translate, Rotate, Scale, Affine, Homography, Homogeneous coordinates)
- Viewing and Projection (matrix composition, perspective, Z-buffer)
- Shader Pipeline (Graphics Processing Unit)
- Animation (kinematics, keyframing, Catmull-Romm interpolation, physical simulation)
- Mass Springs simulation (forces, particles, simulation)

What is Computer Graphics?

Computers:

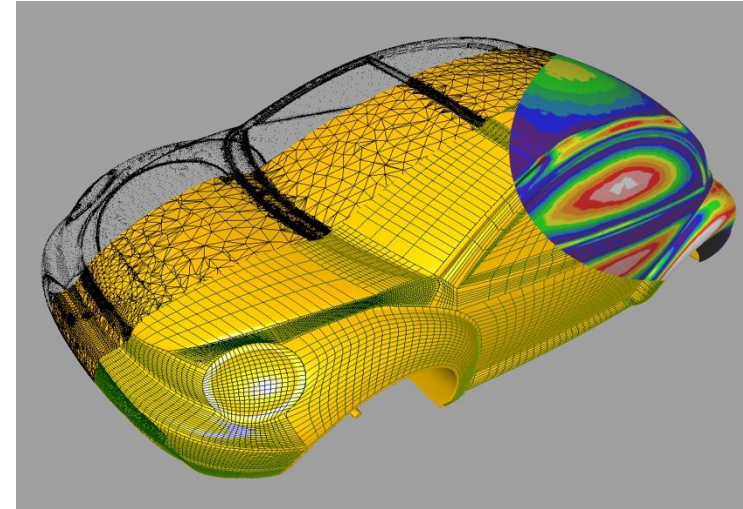
accept, process, transform and present information.

Computer Graphics:

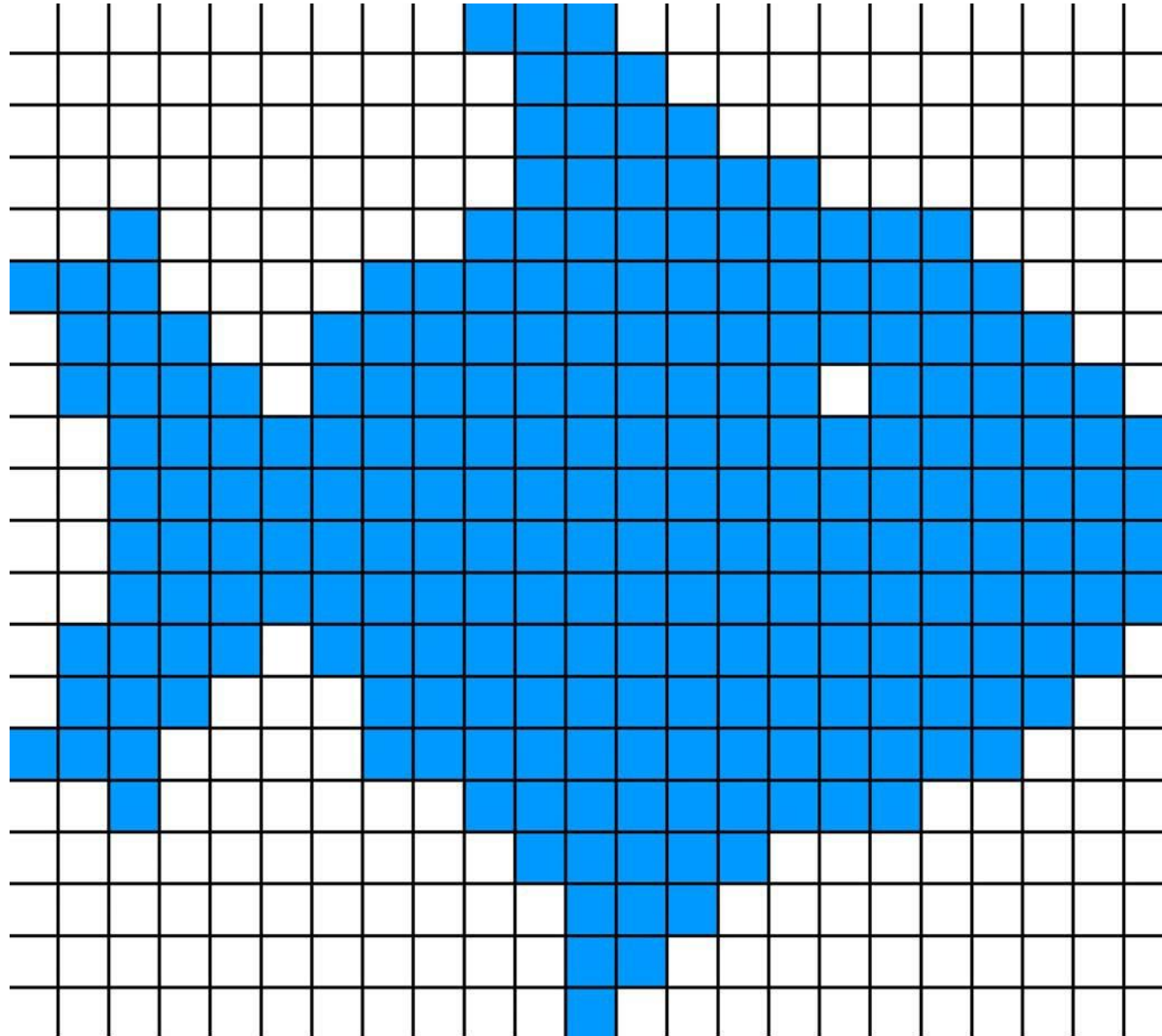
accept, process, transform and present information
in a visual form.

“Core” Areas of Computer Graphics

- Form (modeling)
How do we represent (2D or 3D) objects & scenes?
How do we build these representations?
- Function, Behavior (animation)
How do we represent the way objects move?
How do we define & control their motion?
- Appearance (rendering)
How do we represent the appearance of objects?
How do we simulate the image-forming process?



Raster Image



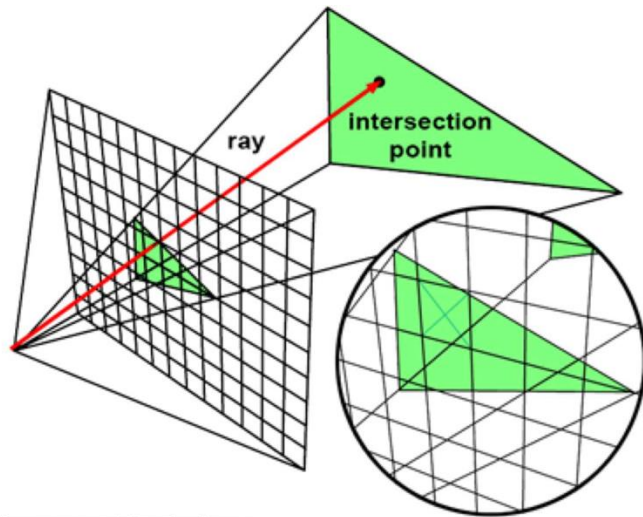
Ray Casting



Ray Casting vs. Rasterization

Ray Casting

```
for each image pixel {  
    for each scene object {  
        ...  
    }  
}
```

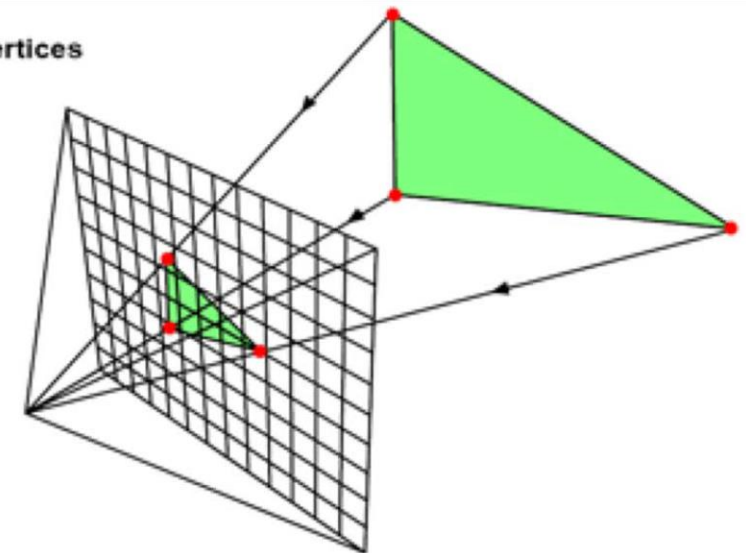


© www.scratchapixel.com

Rasterization

```
for each scene object {  
    for each image pixel {  
        ...  
    }  
}
```

1) Project vertices



© www.scratchapixel.com

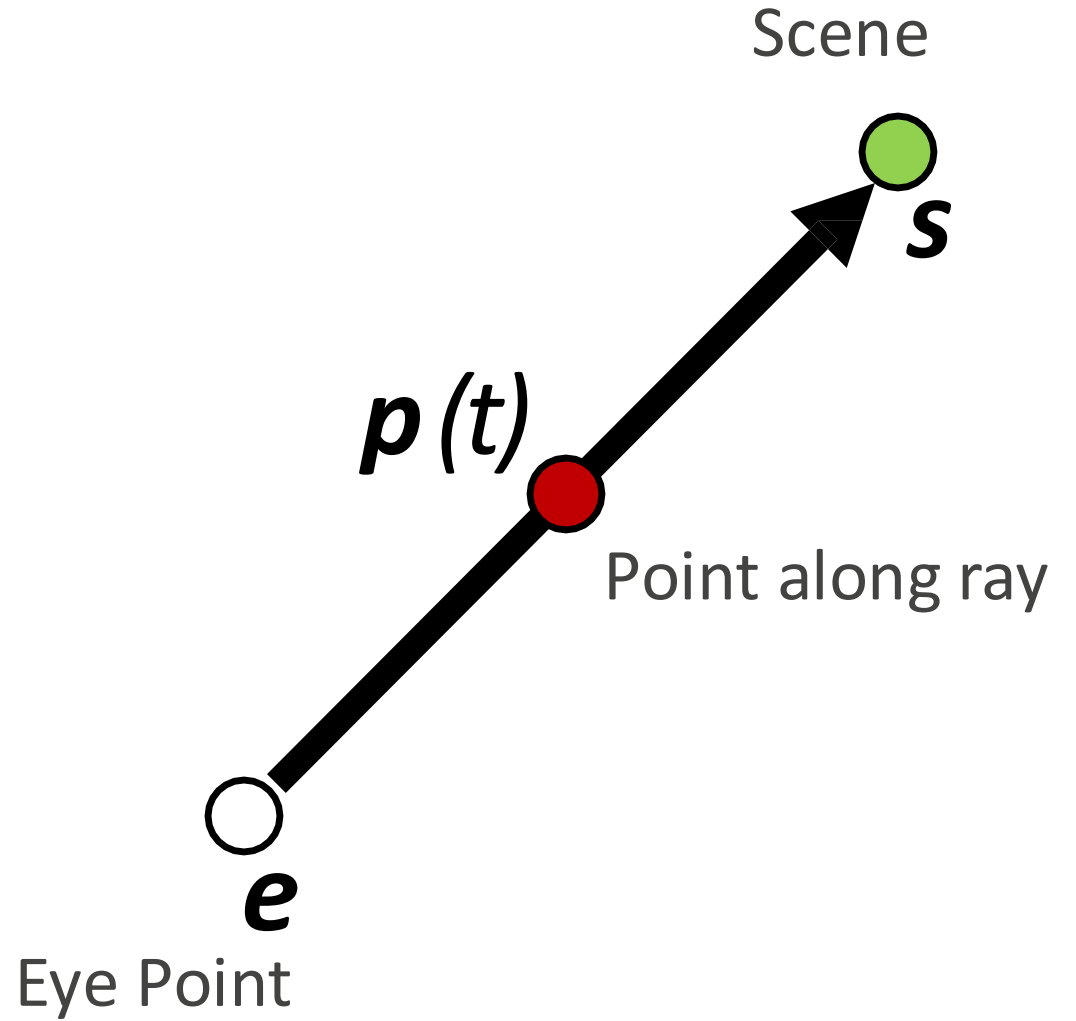


Ray

$$p(t) = e + t(s - e)$$

$$p(0) = e$$

$$p(1) = s$$

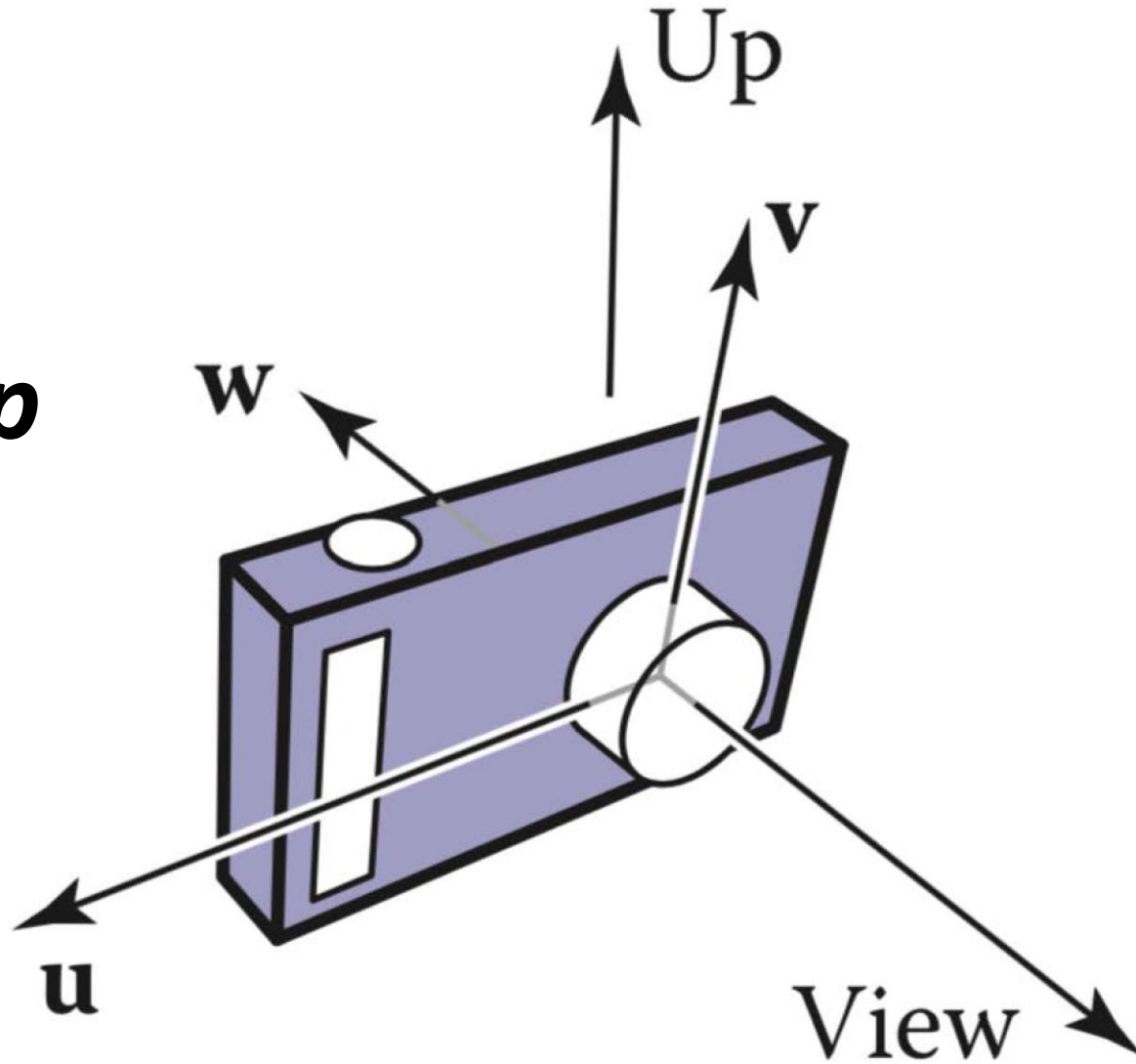


Camera

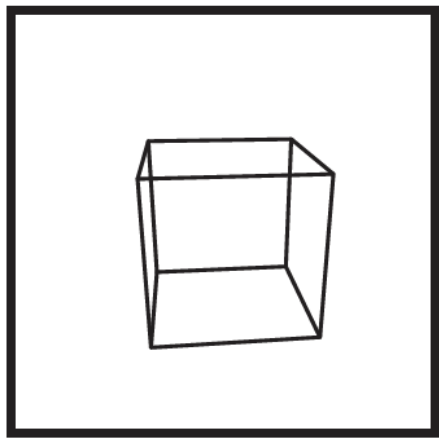
$$\mathbf{w} = -\mathbf{view}$$

$$\mathbf{u} = \mathbf{view} \times \mathbf{up}$$

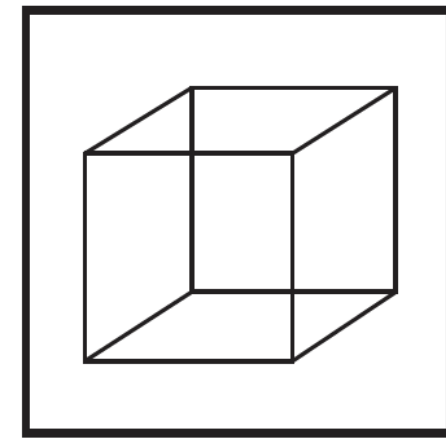
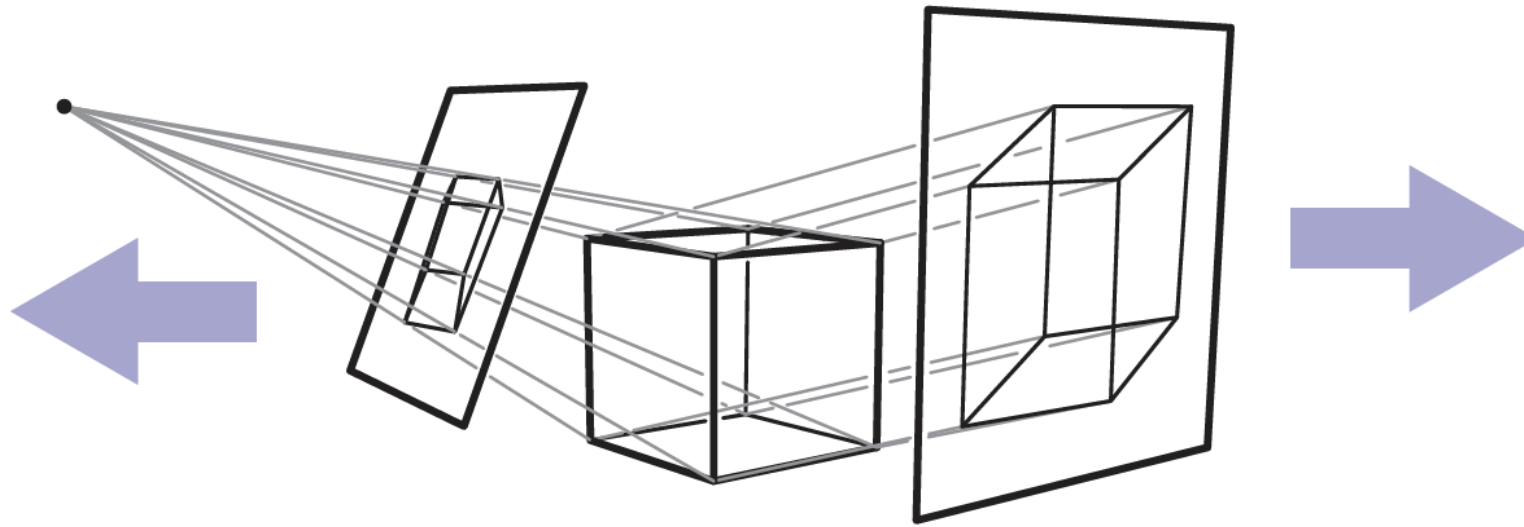
$$\mathbf{v} = \mathbf{w} \times \mathbf{u}$$



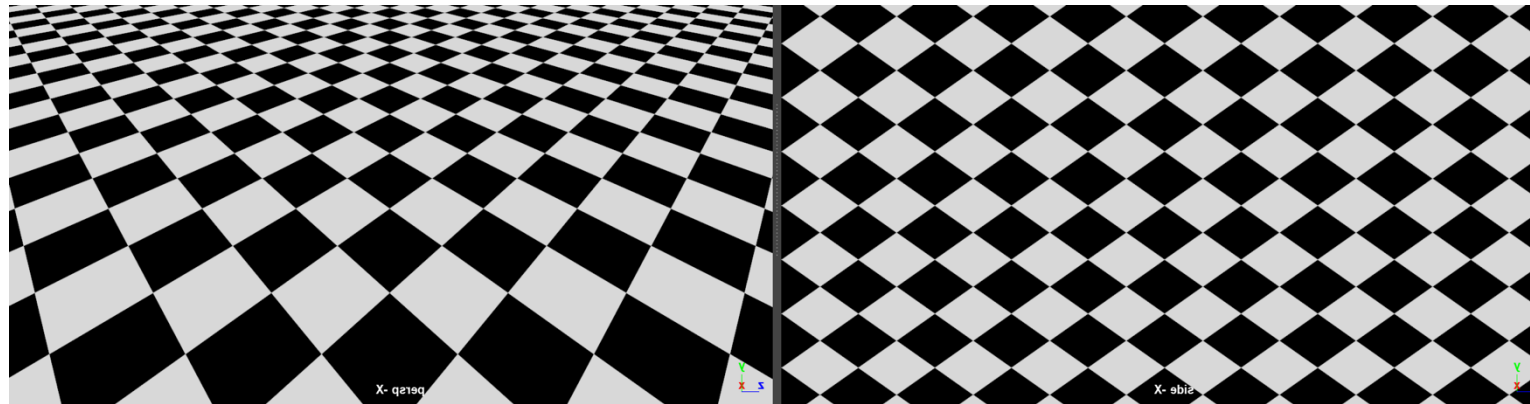
Orthographic v Perspective Projection



Perspective



Oblique



Ray-Sphere Intersection

Substitute ray equation into implicit equation for sphere

$$(\mathbf{e} + t\vec{\mathbf{d}} - \mathbf{c}) \cdot (\mathbf{e} + t\vec{\mathbf{d}} - \mathbf{c}) - r^2 = 0$$

Rearrange

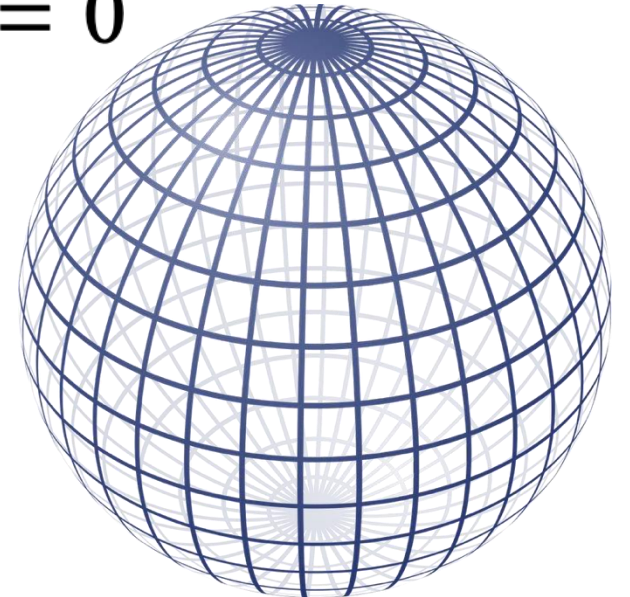
$$(\vec{\mathbf{d}} \cdot \vec{\mathbf{d}})t^2 + 2\vec{\mathbf{d}} \cdot (\mathbf{e} - \mathbf{c})t + (\mathbf{e} - \mathbf{c}) \cdot (\mathbf{e} - \mathbf{c}) - r^2 = 0$$

Looks familiar...

$$At^2 + Bt + C = 0$$

It's a quadratic! (can use the quadratic equation)

Hint: the discriminant tells us what kinds of roots the equation has.



Intersection with a Triangle (Parametric Surface)

Check via equating point on surface with point on ray

$$\mathbf{p}(t) = \alpha \mathbf{t}_1 + \beta \mathbf{t}_2$$

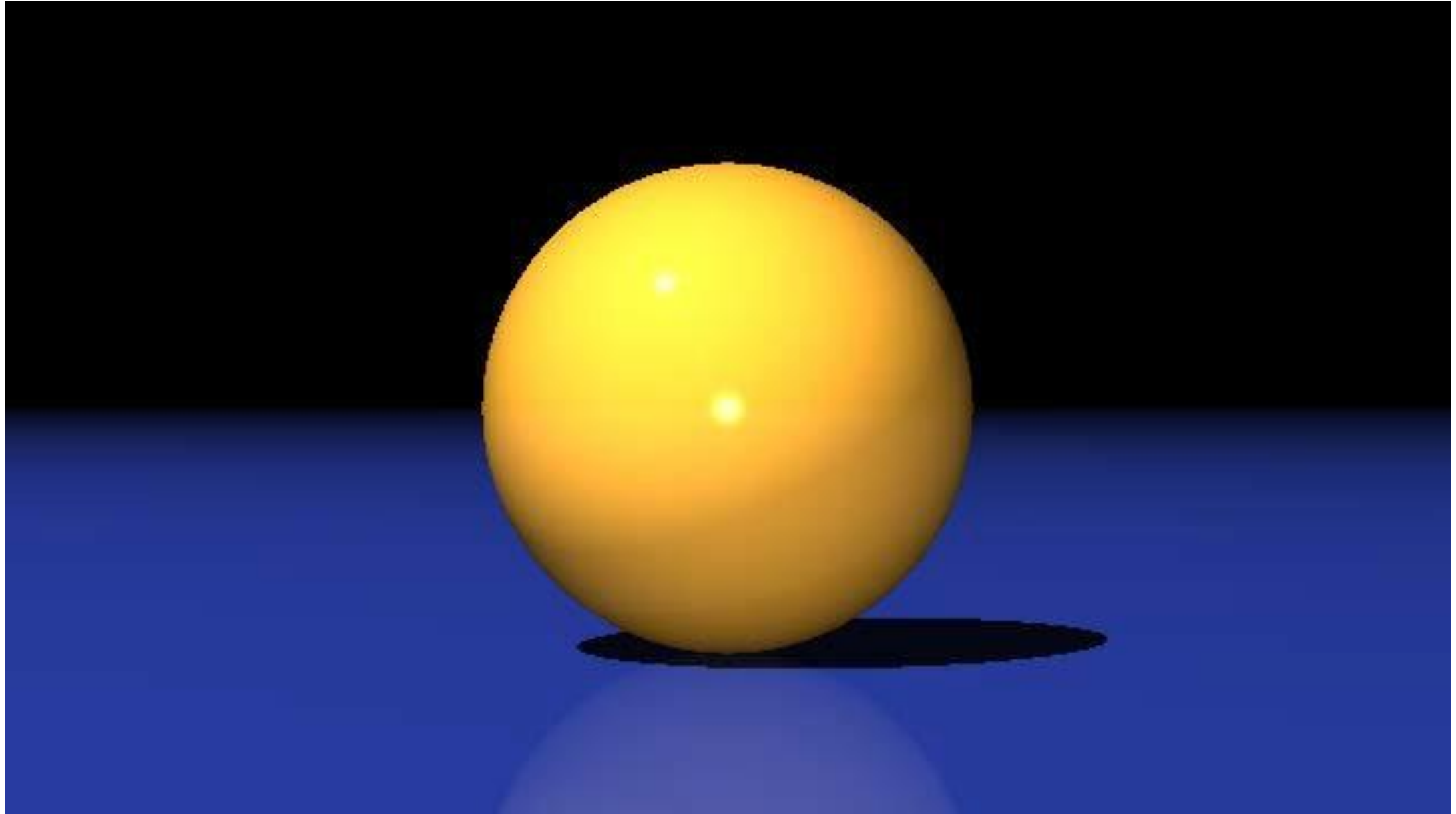
$$\mathbf{e} + t\vec{\mathbf{d}} = \alpha \mathbf{t}_1 + \beta \mathbf{t}_2$$

$$\mathbf{e} = \alpha \mathbf{t}_1 + \beta \mathbf{t}_2 - t\vec{\mathbf{d}}$$

$$\mathbf{e} = \begin{bmatrix} \mathbf{t}_1 & \mathbf{t}_2 & -\mathbf{d} \end{bmatrix} \begin{bmatrix} \alpha \\ \beta \\ t \end{bmatrix}$$

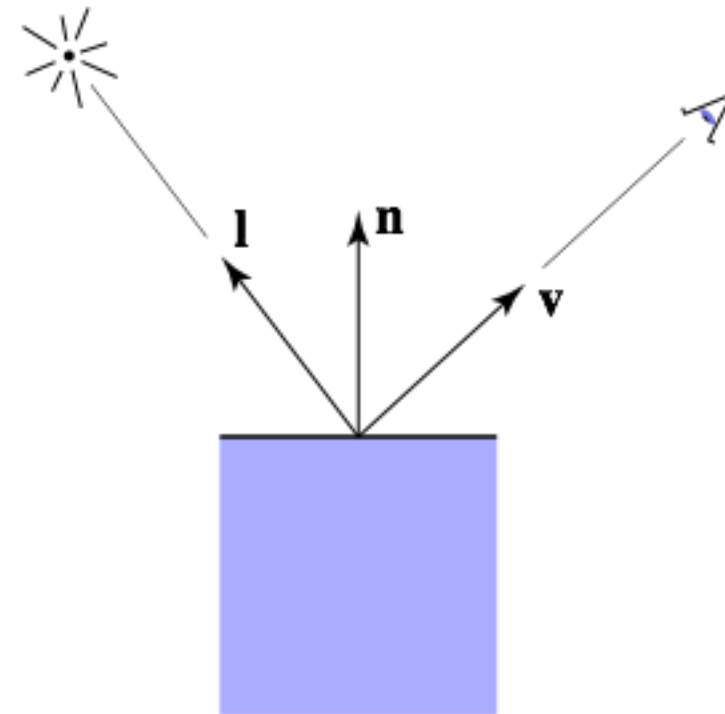


Ray Tracing



Shading

- Compute light reflected toward camera
- Inputs:
 - eye direction
 - light direction
(for each of many lights)
 - surface normal
 - surface parameters
(color, shininess, ...)



Computing the Normal at a Hit Point

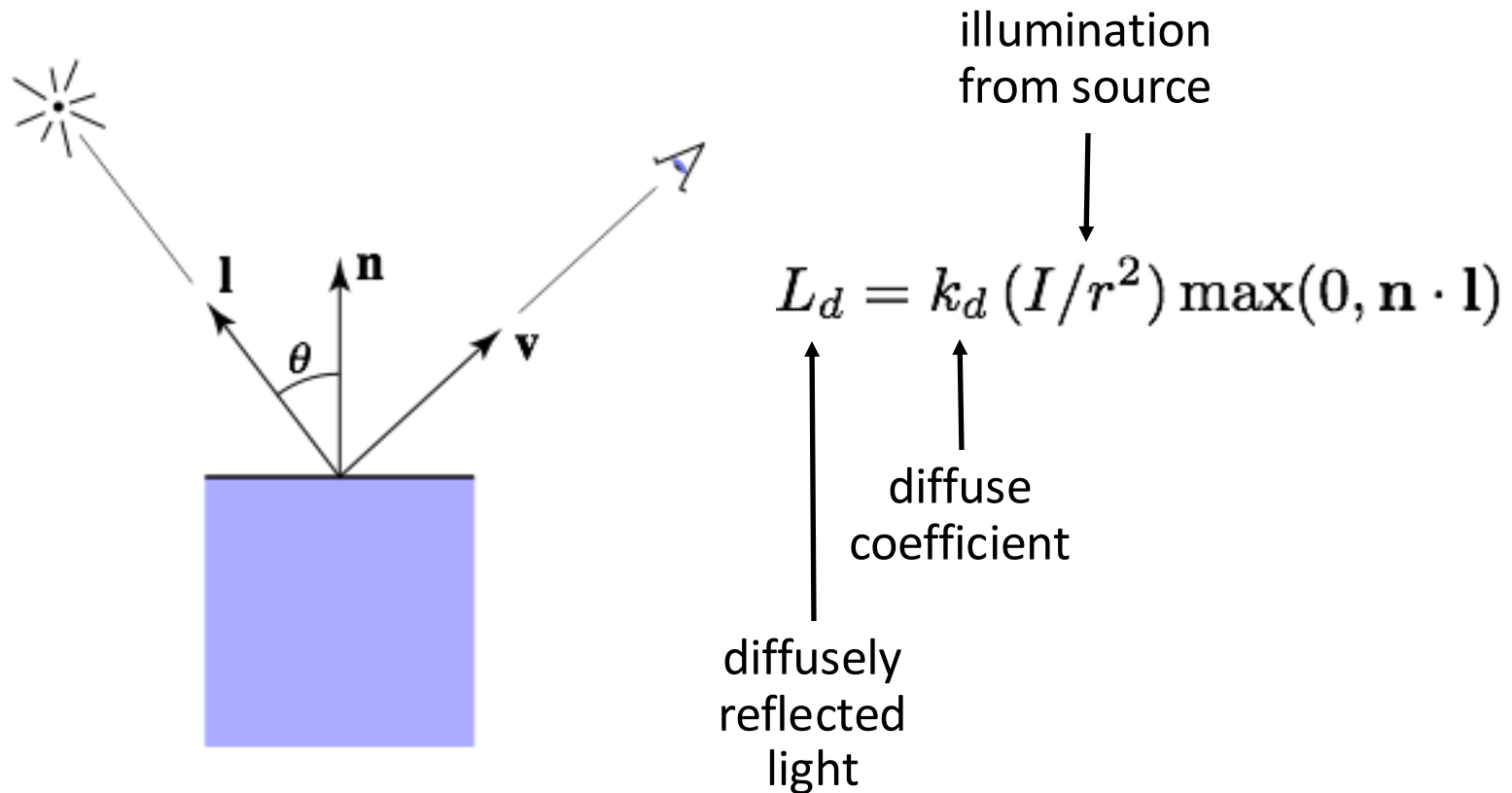
- Polygon normal: cross product of two non-collinear edges.
- Implicit surface normal $f(p)=0$:
 $\text{gradient}(f)(p).$
- Explicit parametric surface $f(a,b)$:

$$\delta f(s,b)/\delta s \times \delta f(a,t)/\delta t.$$



Lambertian shading

Shading independent of view direction

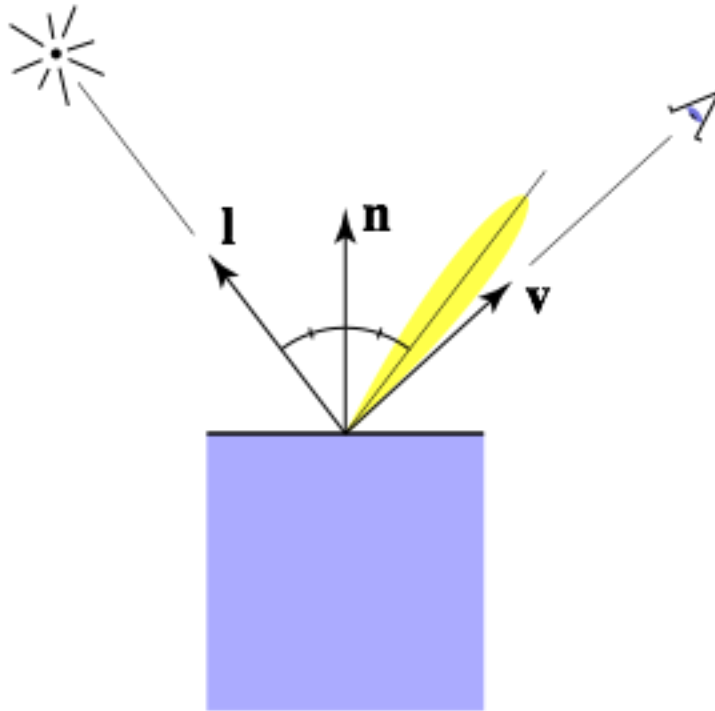


Specular shading (Phong)

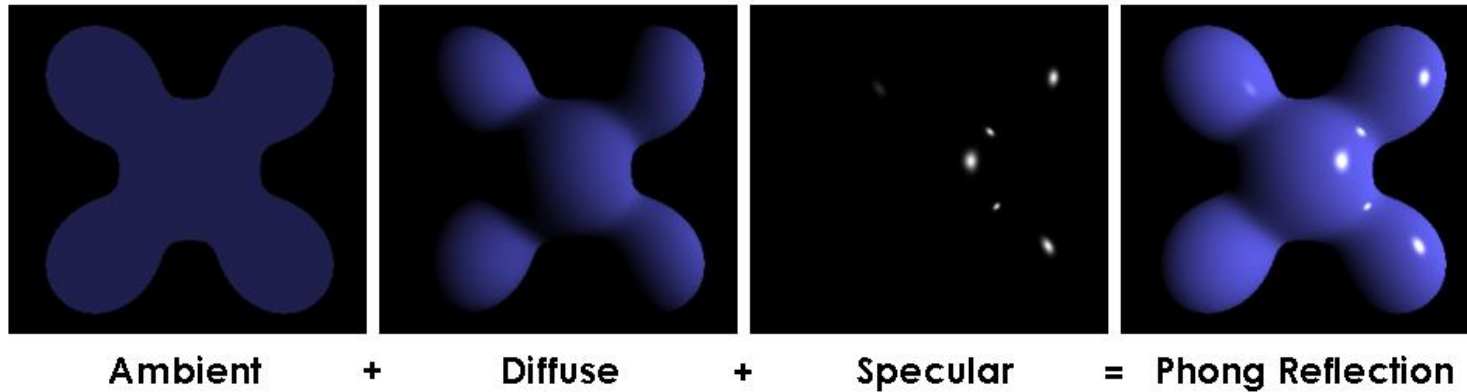
Intensity depends on view direction

– bright near mirror configuration

$$k_s * I_s * (\mathbf{v} \cdot \mathbf{r})^{shiny}$$



Local Illumination



- Usually include ambient, diffuse, Phong in one model

$$\begin{aligned} L &= L_a + L_d + L_s \\ &= k_a I_a + k_d (I/r^2) \max(0, \mathbf{n} \cdot \mathbf{l}) + k_s (I/r^2) \max(0, \mathbf{n} \cdot \mathbf{h})^p \end{aligned}$$

- The final result is the sum over many lights

$$\begin{aligned} L &= L_a + \sum_{i=1}^N [(L_d)_i + (L_s)_i] \\ L &= k_a I_a + \sum_{i=1}^N \left[k_d (I_i/r_i^2) \max(0, \mathbf{n} \cdot \mathbf{l}_i) + \right. \\ &\quad \left. k_s (I_i/r_i^2) \max(0, \mathbf{n} \cdot \mathbf{h}_i)^p \right] \end{aligned}$$



Local vs. Global Illumination

Local Illumination Models

- e.g. Phong, Blinn.
- Model source from a light reflected once off a surface towards the eye.
- Indirect light is included with an ad hoc “ambient” term which is normally constant across the scene.

Global Illumination Models

- e.g. recursive ray tracing (incomplete model).
- Try to measure light propagation in the scene.
- Model interaction between objects, other objects, and their environment

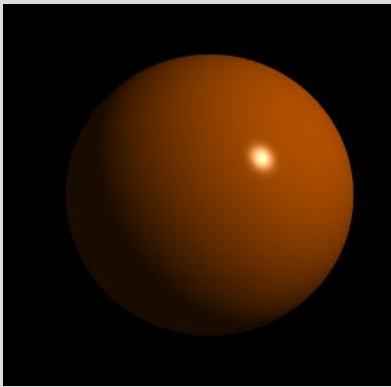


Path Tracing

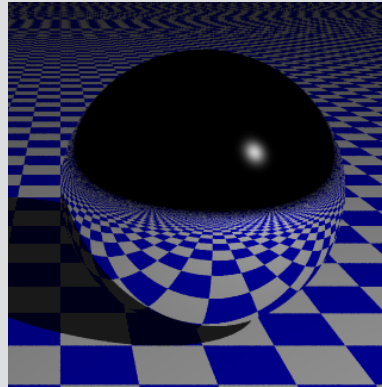
- A ray from a light L can bounce of any number of specular S and diffuse objects D before entering the eye E. The paths from E to L for eg. can be
E-D-L, E-S-D-D-S-S-D-S-L, E-D-D-S-D-S-L...
- Rays are infinitely thin
- Don't disperse
- Ray Tracing model shiny objects exhibiting multiple reflections, i.e. paths of the form
E - S* - D+ - L.



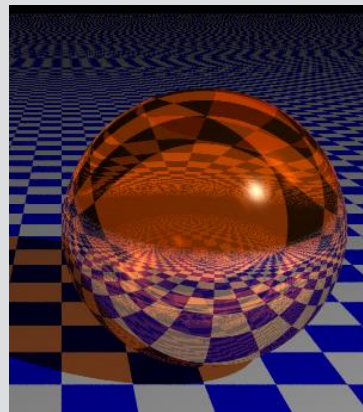
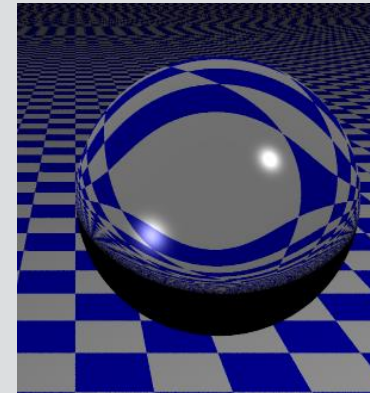
local illumination



reflection

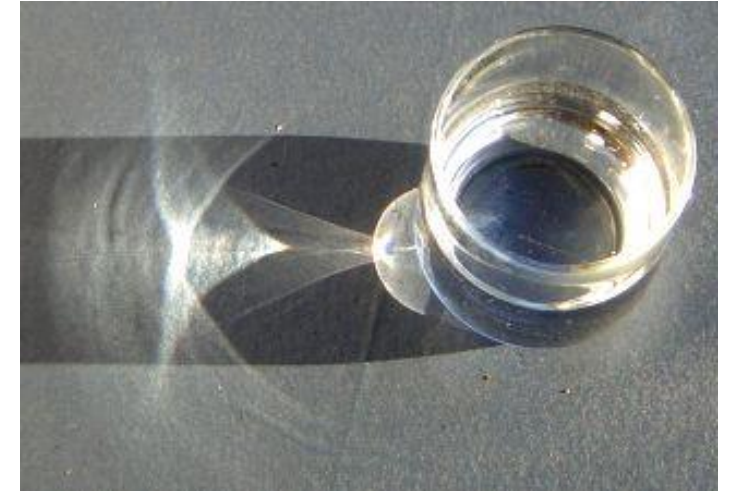


refraction



Ray Tracing Improvements: Caustics

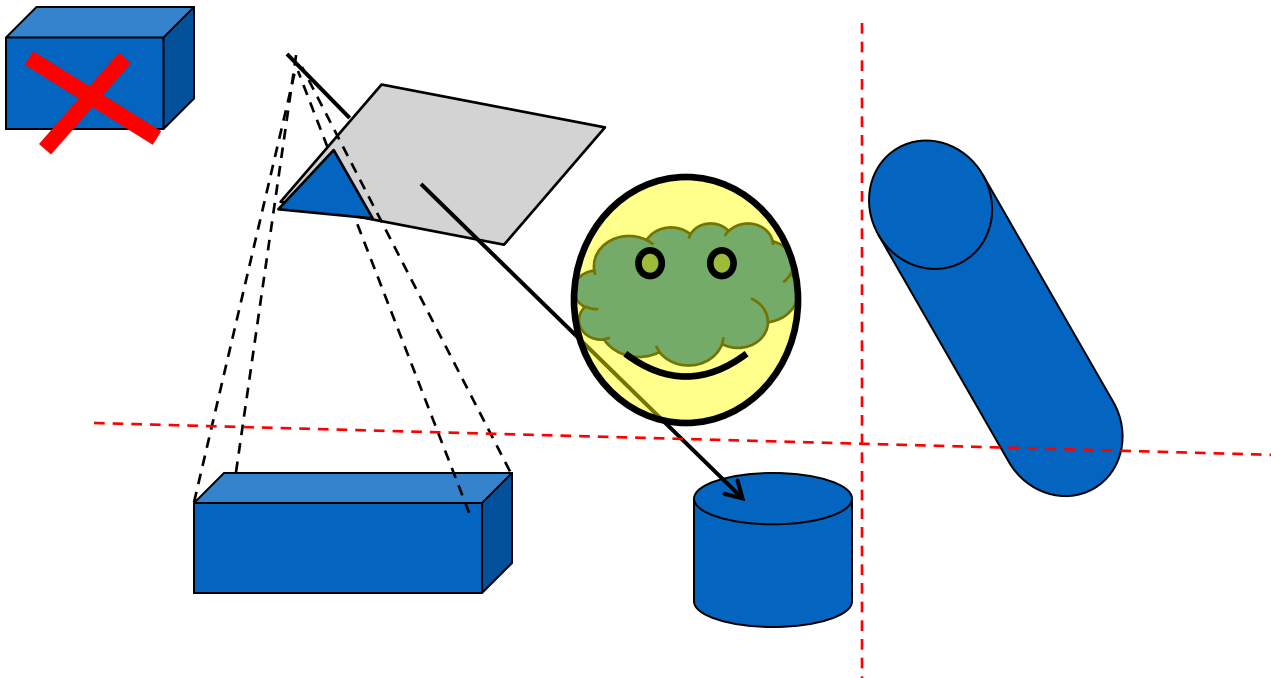
- Transport $E - S - S - S - D - S - S - S - L$
- Trace from the light to the surfaces and then from the eye to the surfaces
- “shower” scene with light and then collect it
- “Where does light go?” vs “Where does light come from?”
- Good for caustics



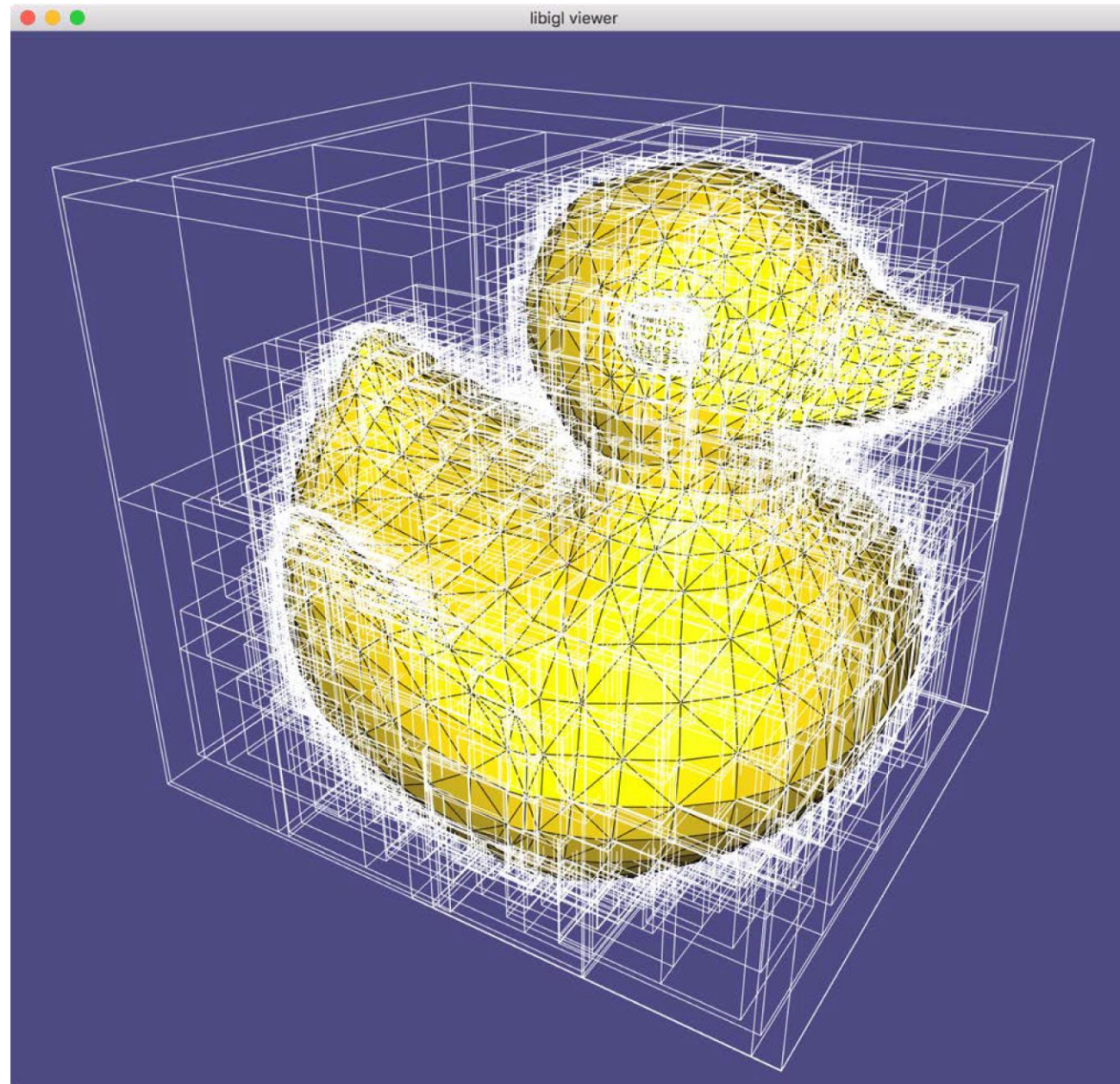
Ray Intersection: Efficiency Considerations

Speed-up the intersection process.

- Ignore object that clearly don't intersect.
- Use proxy geometry.
- Subdivide and structure space hierarchically.
- Project volume onto image to ignore entire sets of rays.



Boundary Volume Hierarchies



Bounding volumes

Quick way to avoid expensive testing intersections and collisions:

 bound object with a simple volume **bvol** (volume encloses object)

If a ray/object doesn't hit/intersect **bvol**,

 it doesn't hit/intersect the object

else

 test object for hit/intersect

Cost: more for hits and near misses, less for far misses

Worth doing? Yes if:

 Cost of **bvol** intersection test is small: simple shapes (spheres, boxes, ...)

 Cost of object intersect test is large: **bvol** most useful for complex objects

 Tightness of fit is good:

 Loose fit leads to extra object intersections

 Tradeoff between tightness and **bvol** intersection cost



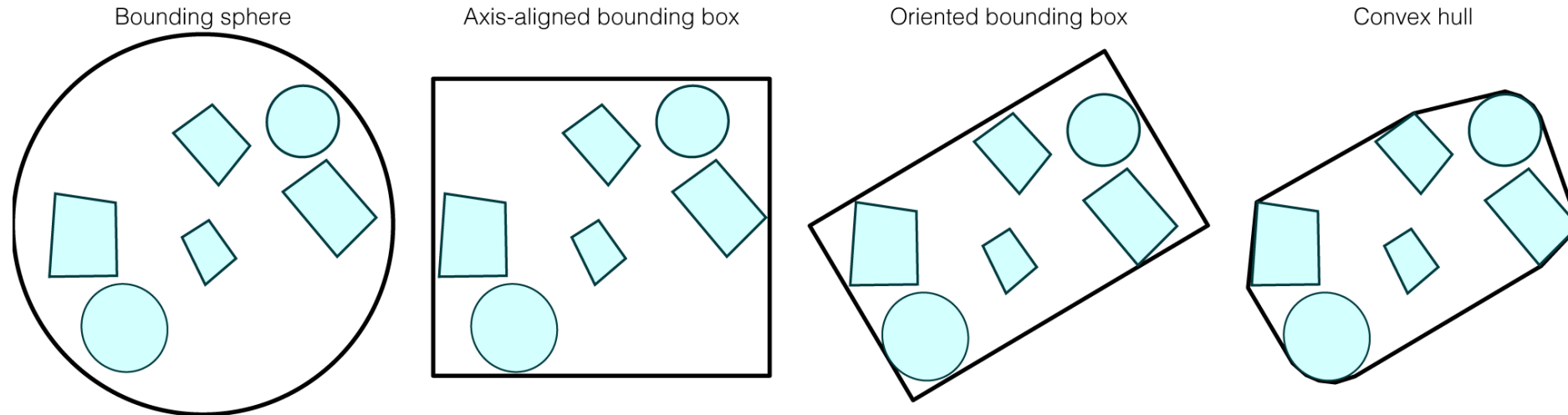
Choice of bounding volumes

Spheres: easy to intersect, not always tight.

Axis-aligned Bounding Boxes (AABBs): easy to intersect, tighter for axis-aligned objects.

Oriented bounding boxes (OBBs): easy to intersect (transformation cost),
tighter than AABBs.

Convex Hull: not as easy to intersect as the above, tighter than the above.

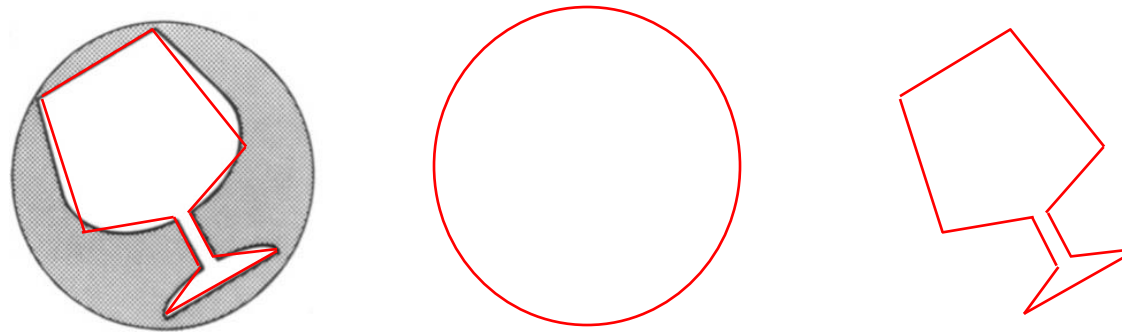


Proxy Geometry vs. Bounding volumes

Another concept often used in CG is proxy geometry or Level-Of-Detail LOD.

A proxy is a simplified representation of the object, that can be used as the object when rendering and processing speed is more important than visual accuracy.

Note, that proxy geometry is an approximation to the object and typically not a bounding volume. And a bounding volume itself is typically not a good visual proxy for an object.



[Glassner 89, Fig 4.5]

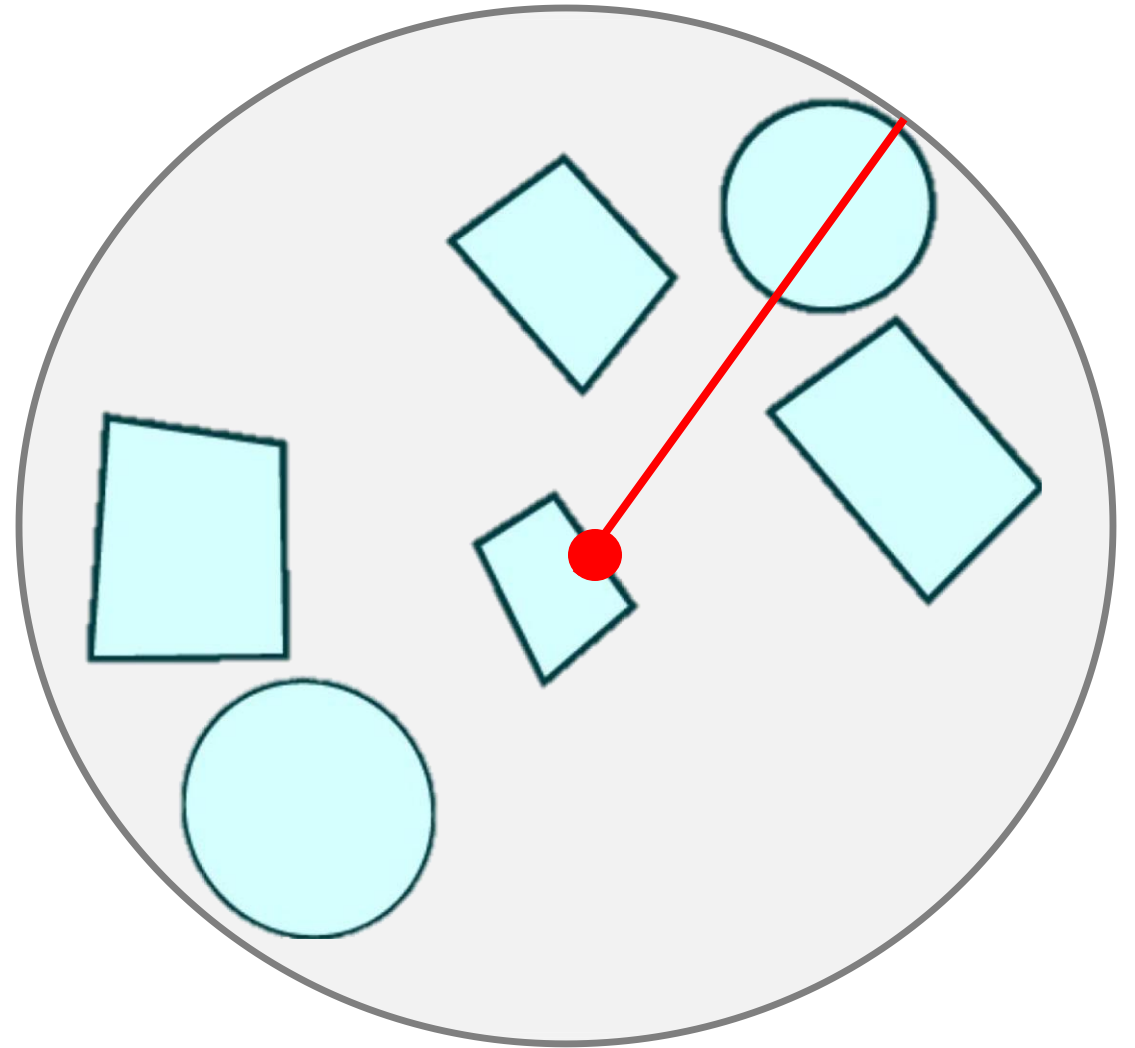


Building a Bounding Sphere

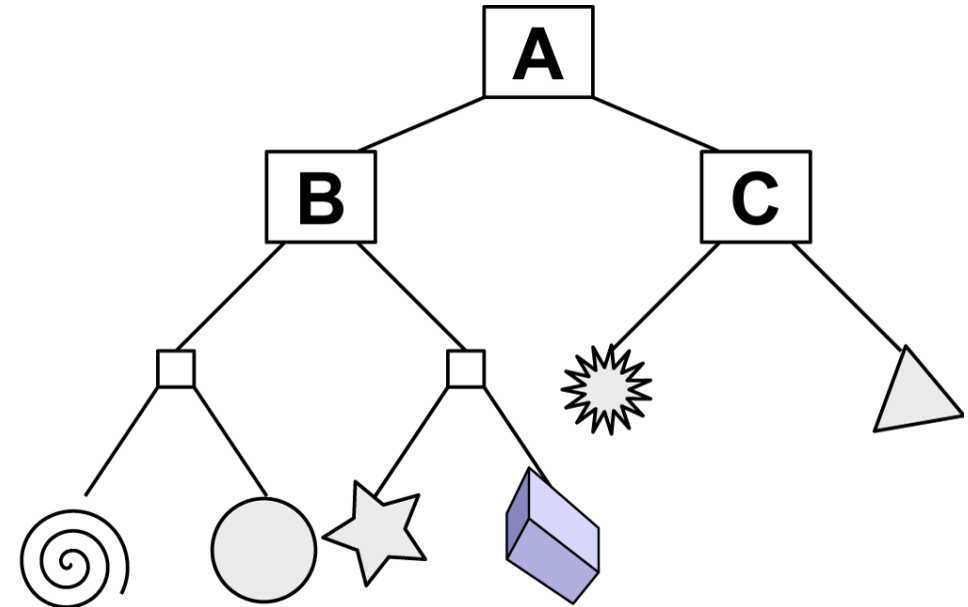
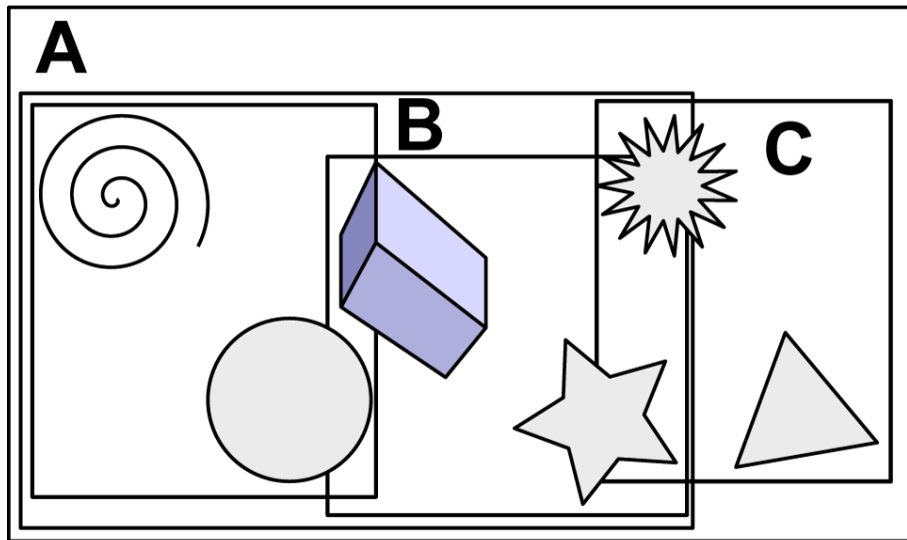
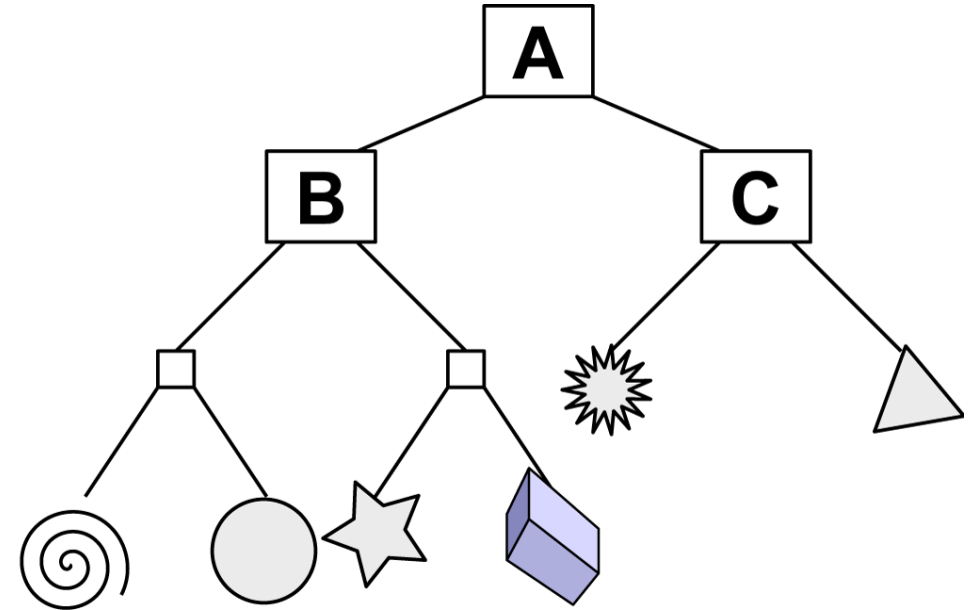
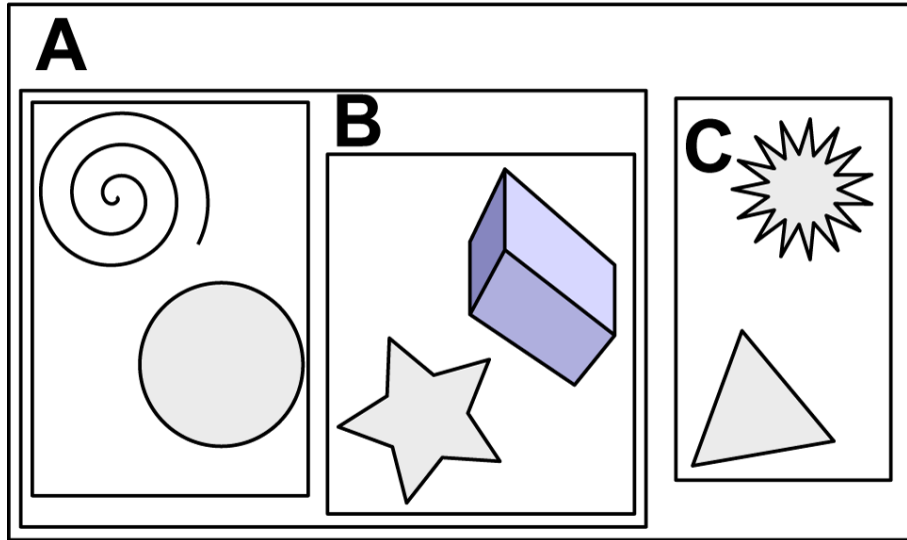
Parameters of a Sphere:

1. Center = $\mathbf{c} = \frac{1}{n} \sum_{i=1}^n \mathbf{v}^i$
2. Radius = $r = \max(\|\mathbf{v}^i - \mathbf{c}\|)$

$\mathbf{v}^i \in \text{Vertices}$



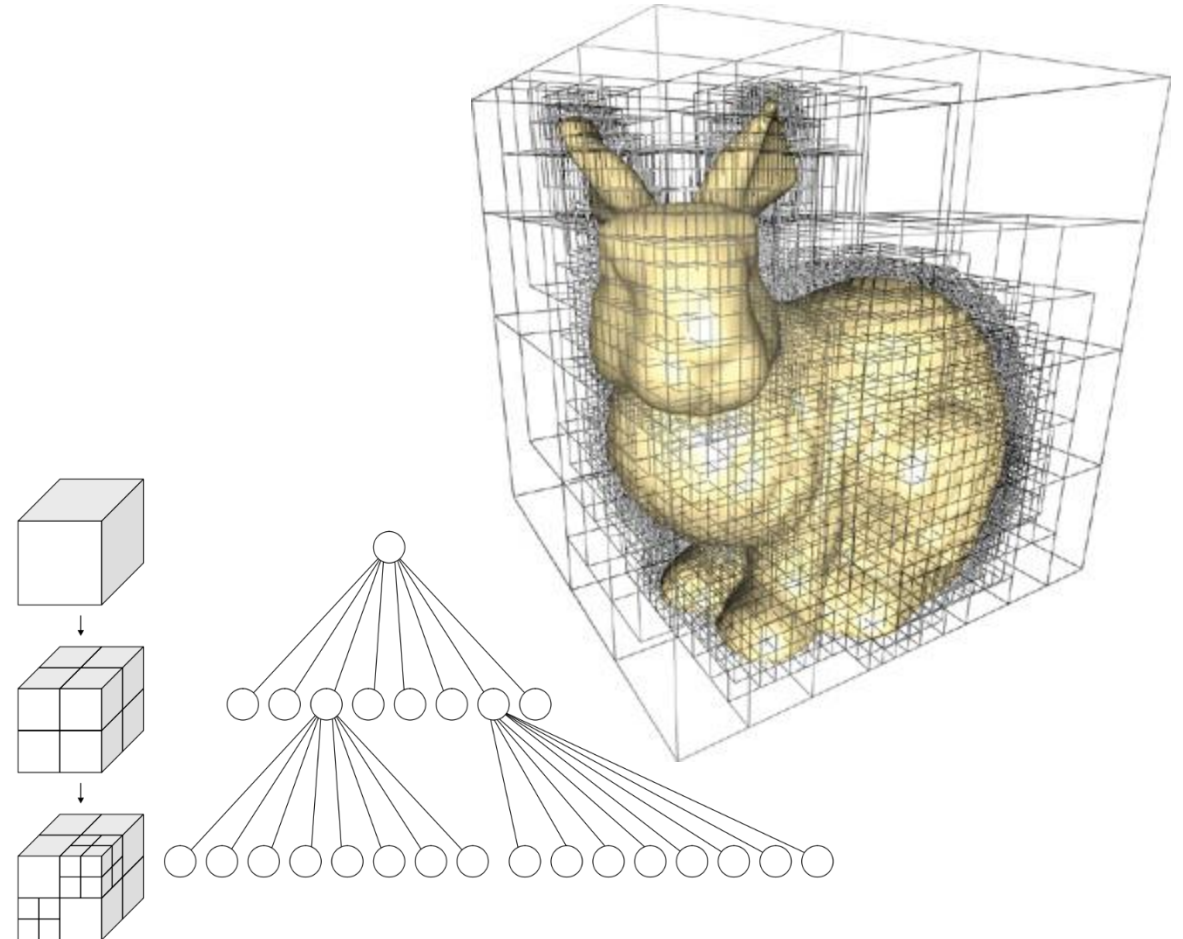
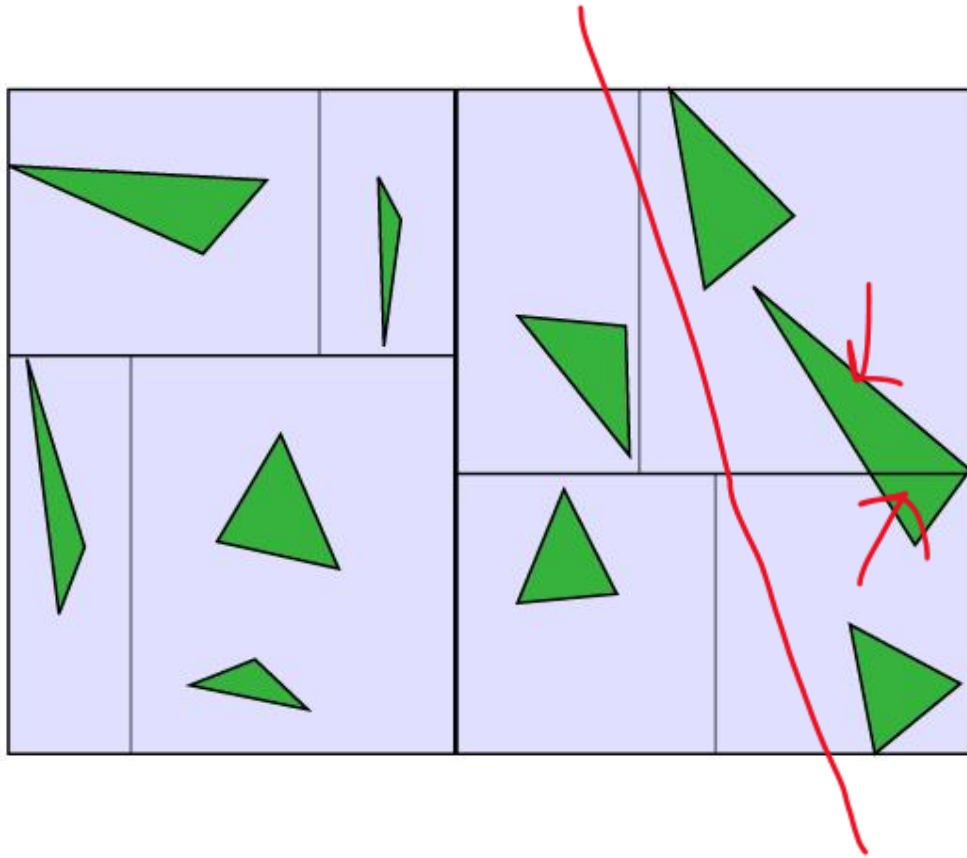
AABB Tree Construction (object based)



Non-regular space subdivision

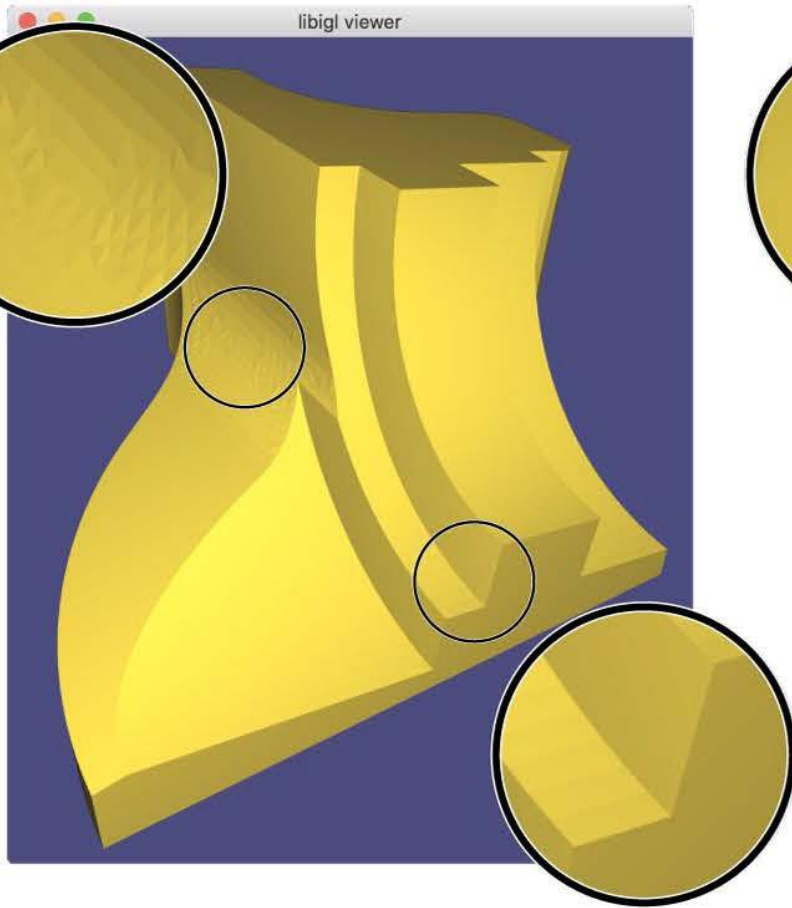
k -d Tree

Octree

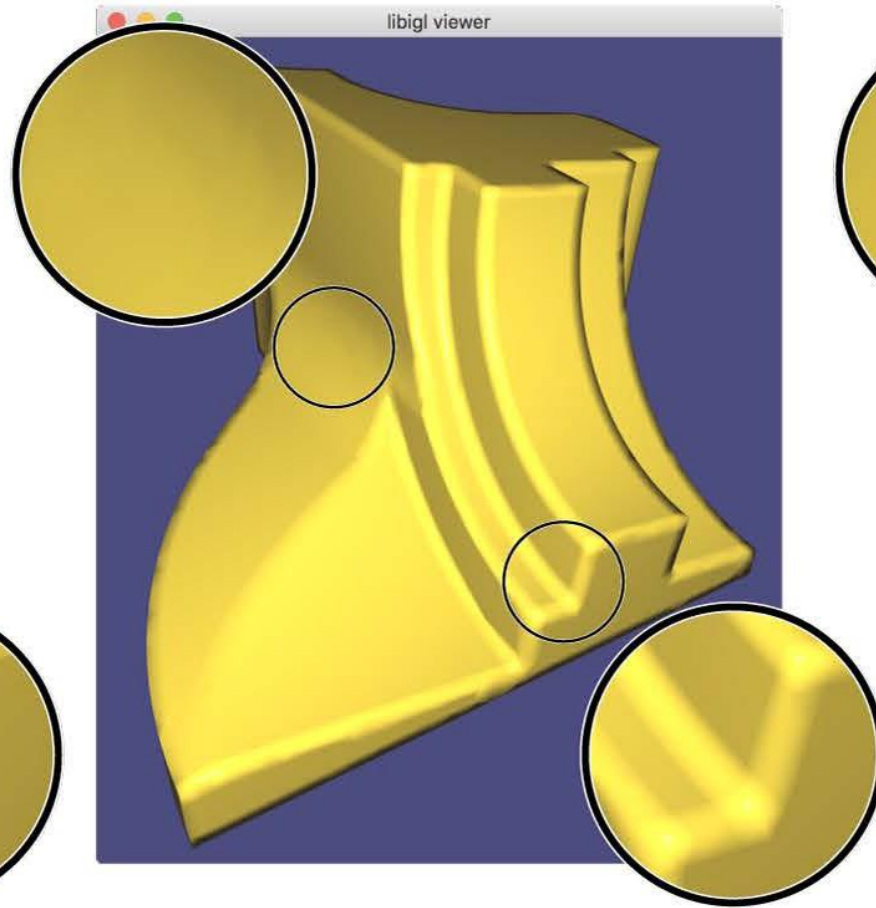


Meshes

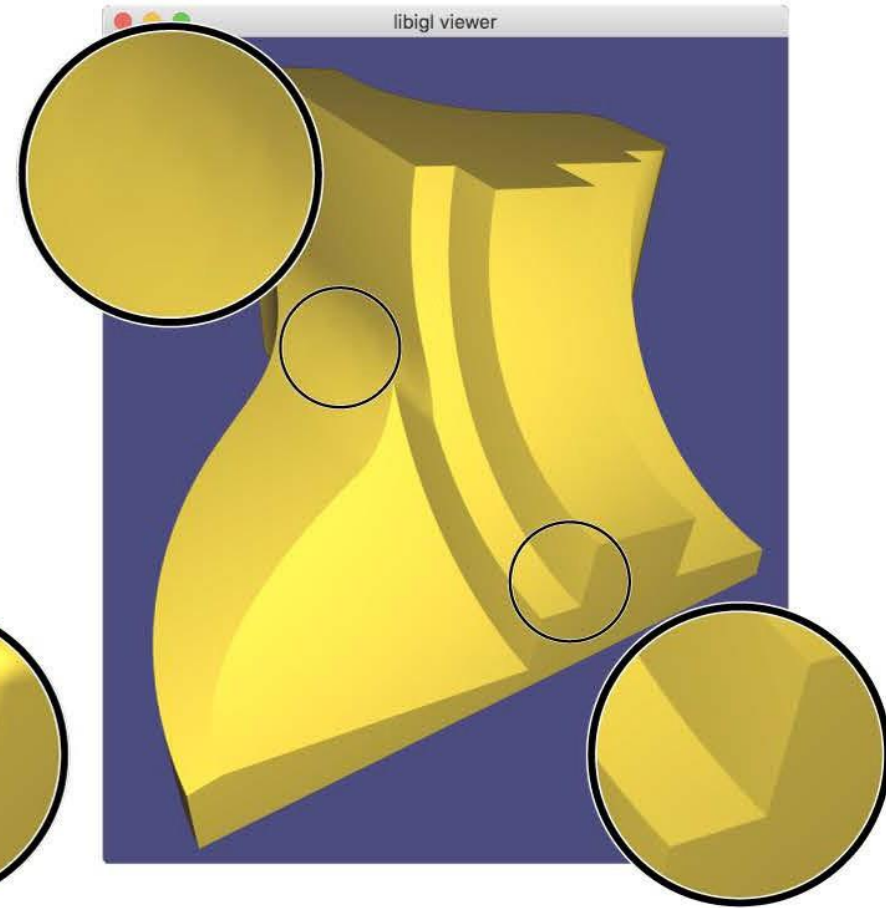
Per-face normals

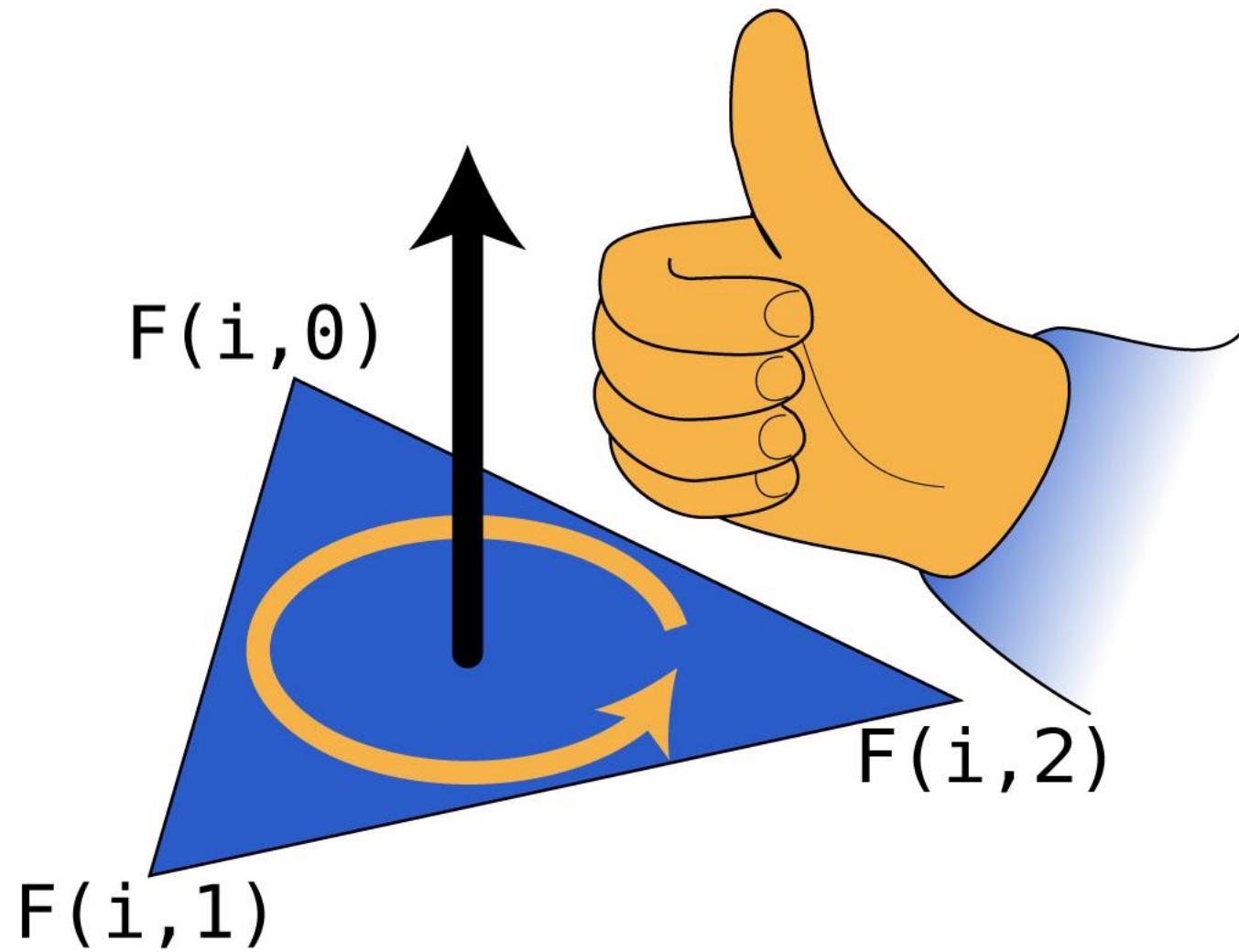


Per-vertex normals



Per-corner normals

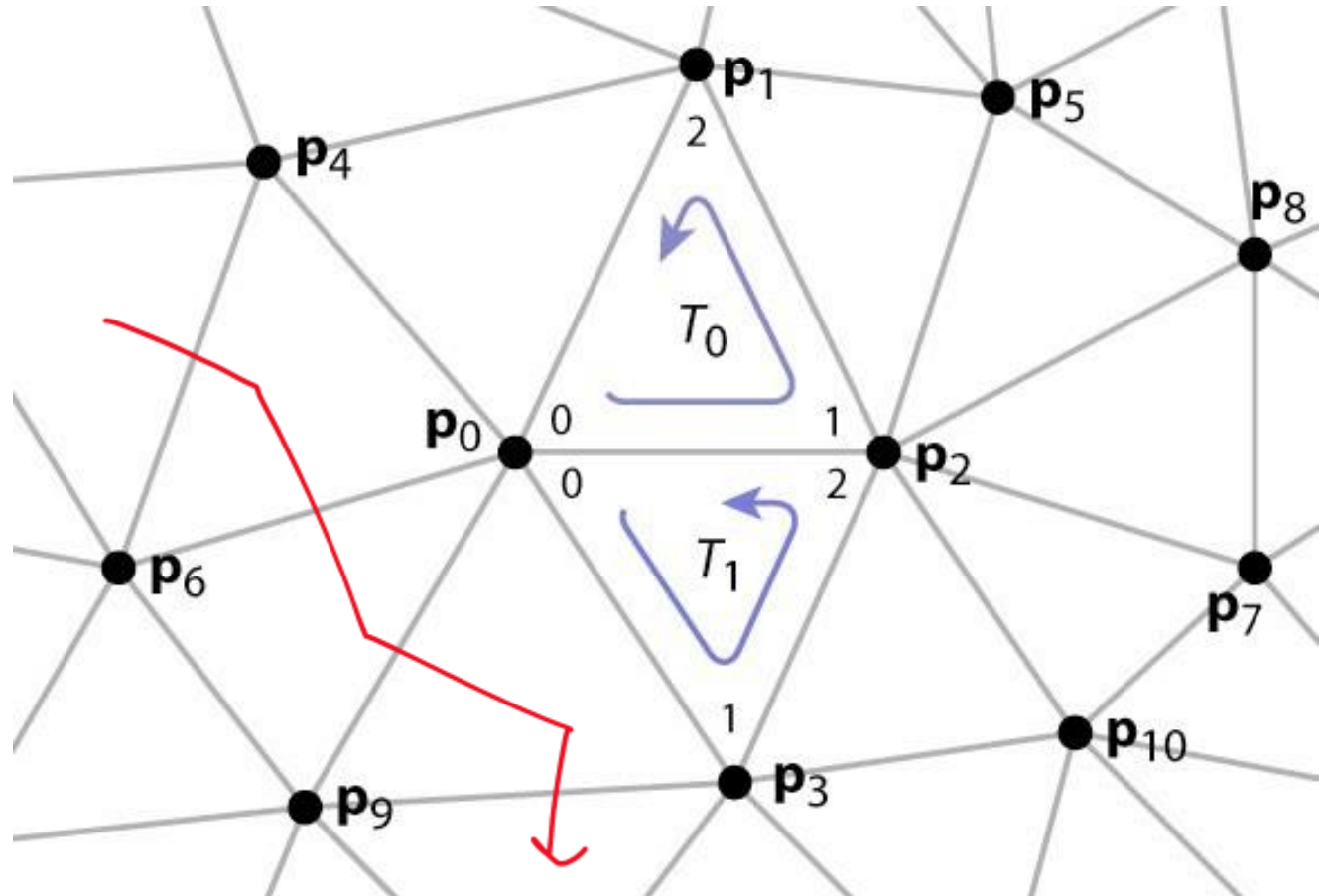




Indexed triangle set

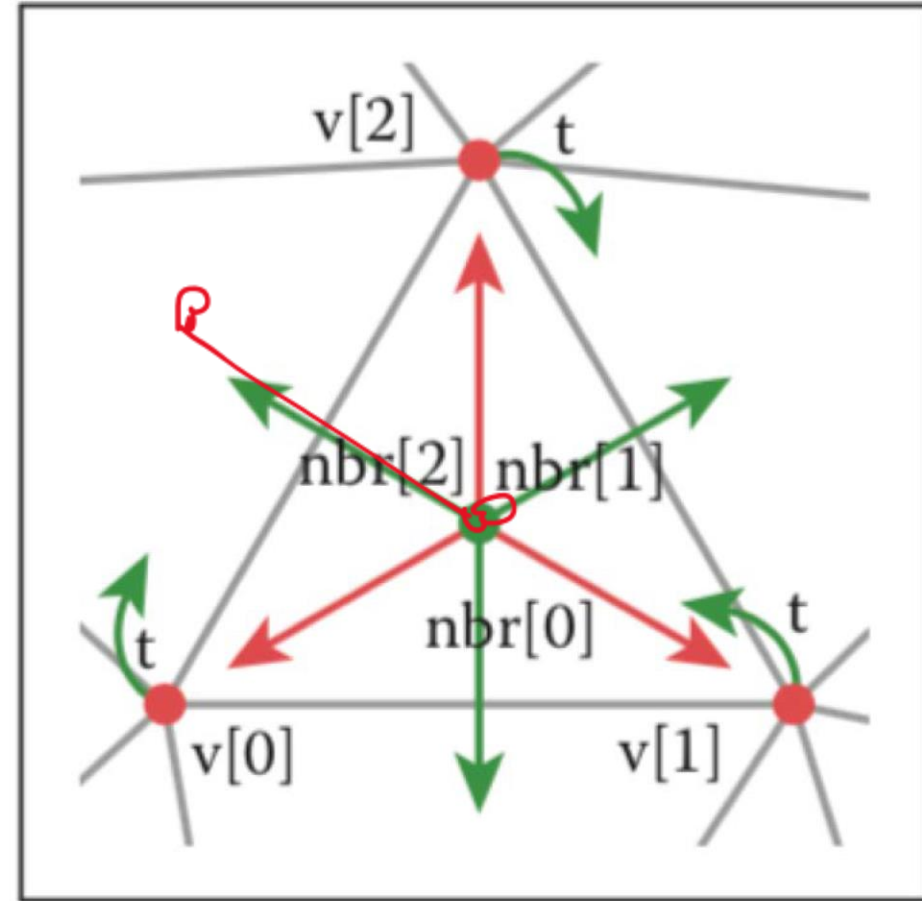
verts[0]	x_0, y_0, z_0
verts[1]	x_1, y_1, z_1
	x_2, y_2, z_2
	x_3, y_3, z_3
	$\vdots$

tInd[0]	0, 2, 1
tInd[1]	0, 3, 2
	$\vdots$

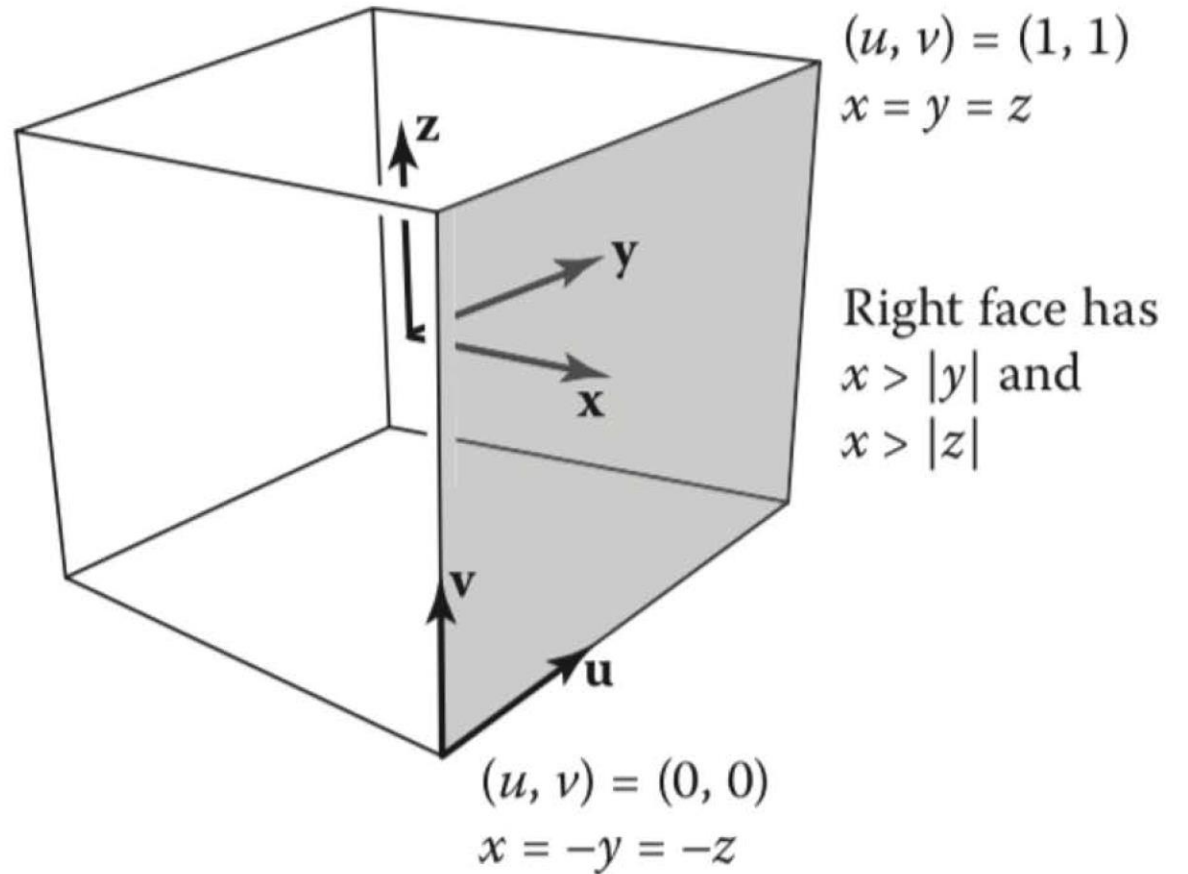
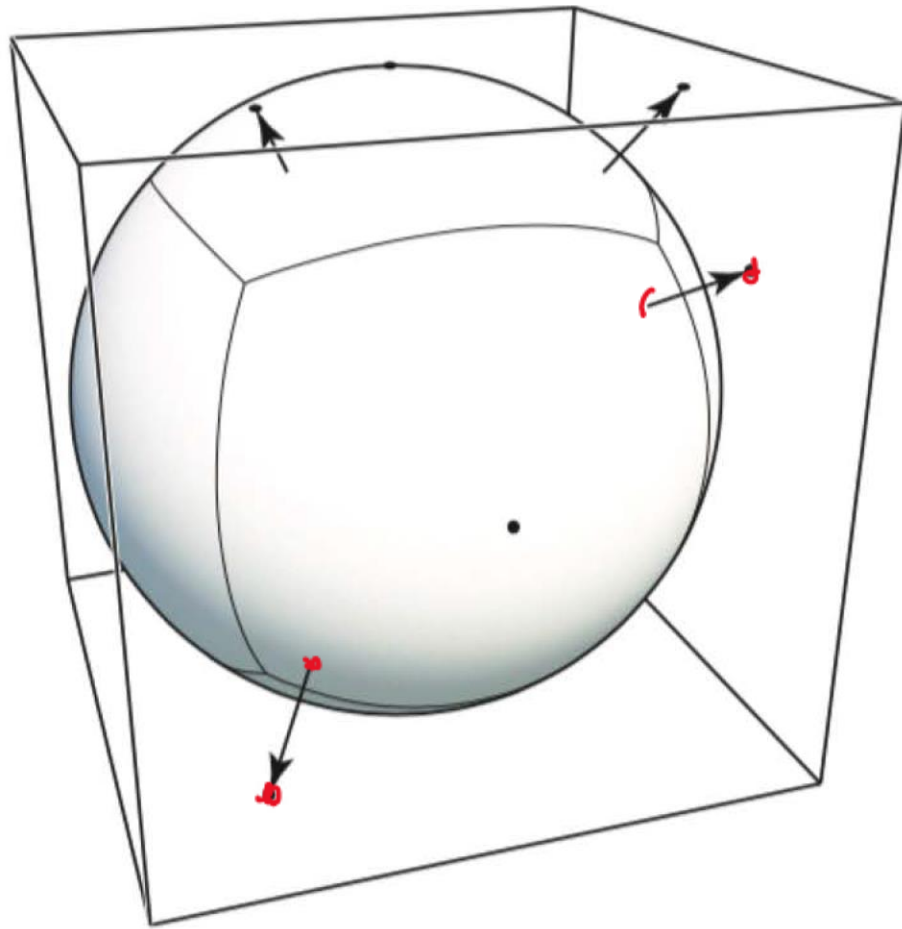


Triangle-Neighbour Data Structure

```
Mesh {  
    float3 verts[nv];    // 3D vertex positions  
    int vTri[nv];         // index of any adjacent triangle  
  
    int tInd[nt][3];      // vertex indices  
    int tNbr[nt][3];      // indices of neighbor triangles  
}
```

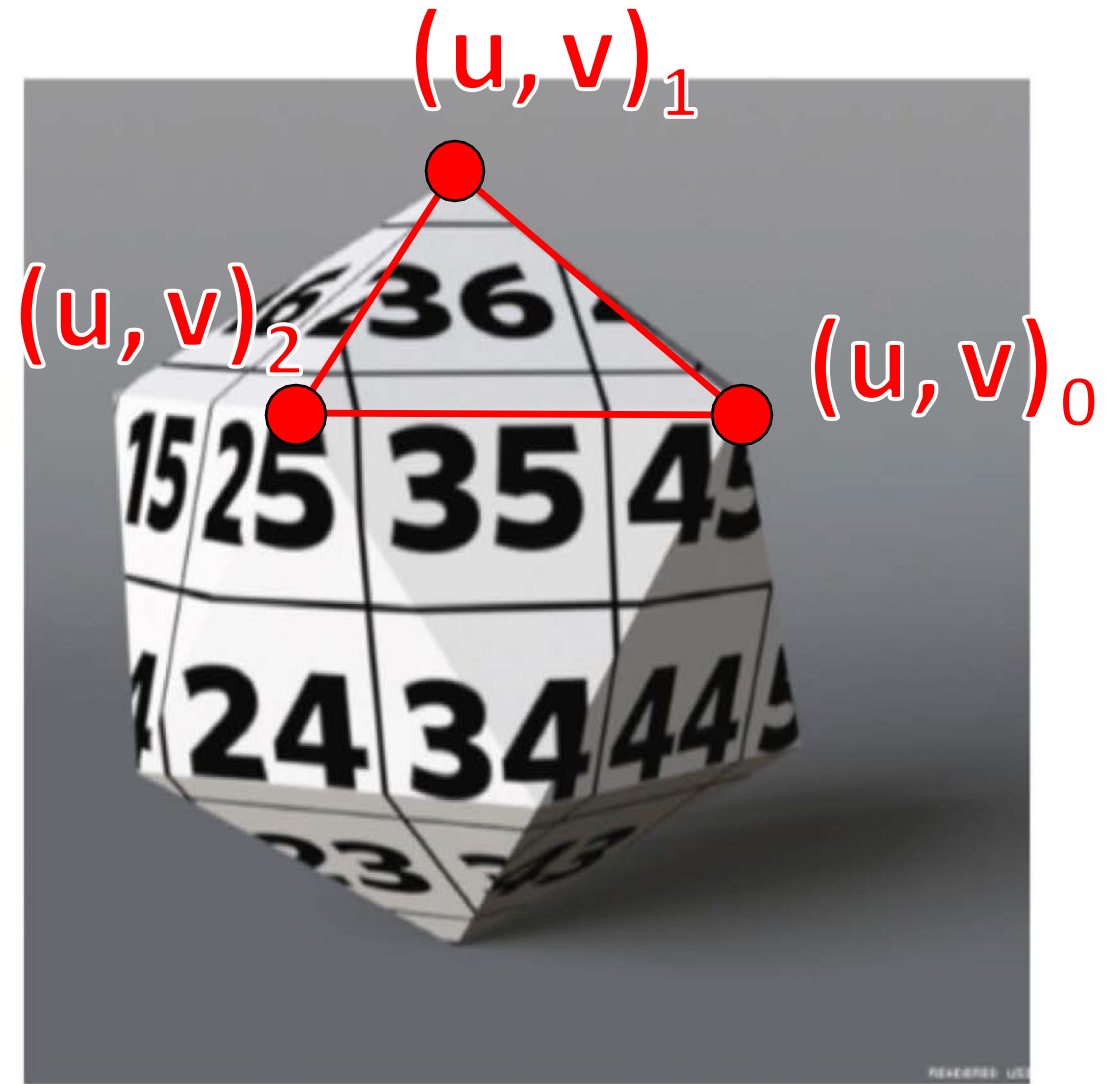


Cube Texture Map



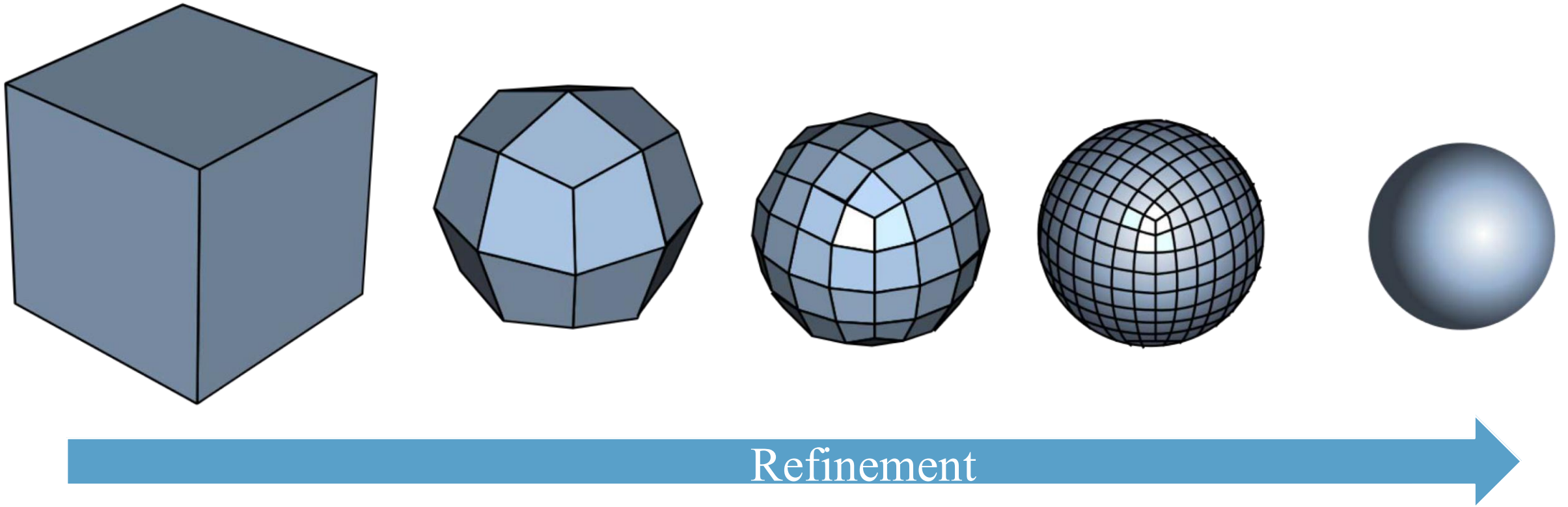
$$(x, y, z) \mapsto \left(\frac{x}{z}, \frac{y}{z} \right).$$

Interpolated Texture Coordinates

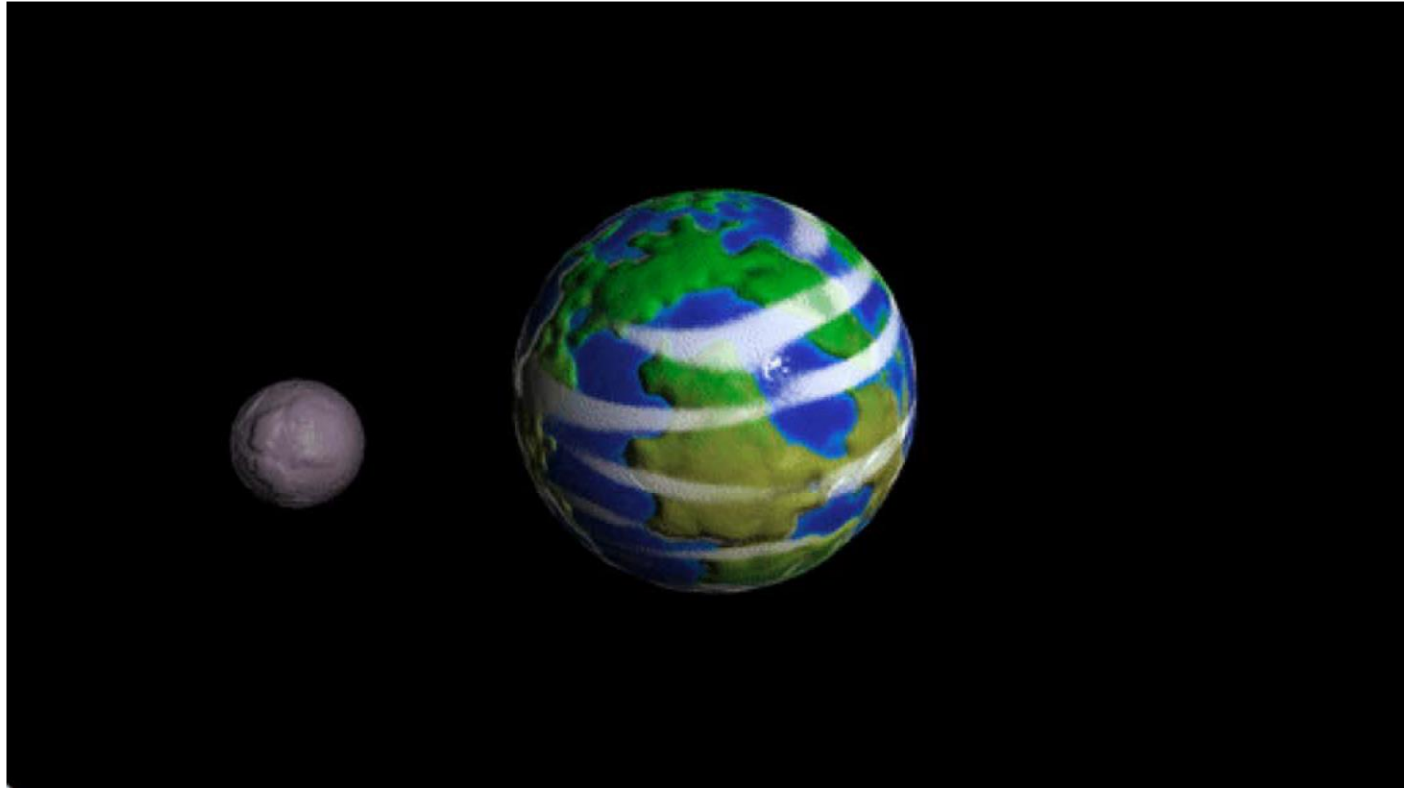


Catmull-Clark Subdivision

Particular type of subdivision scheme.



Shaders



2D Affine Transformations - Translation

$$\begin{bmatrix} a_{11} & a_{12} & t_x \\ a_{21} & a_{22} & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} a_{11}x + a_{12}y + t_x \\ a_{21}x + a_{22}y + t_y \\ 1 \end{bmatrix}$$

$$Ax + t = b$$



Properties of 2D transforms

Homography (preserve lines)

Affine (preserve parallelism)
shear, scale

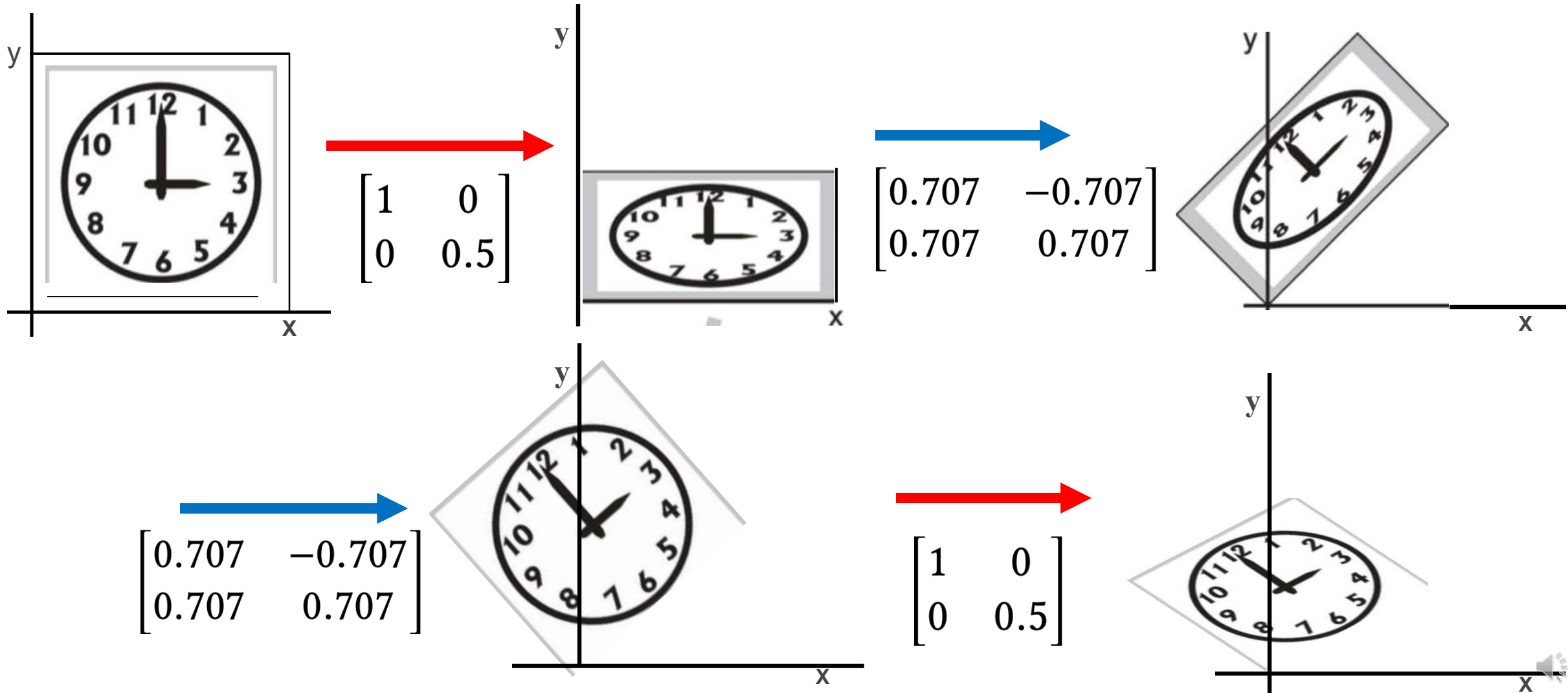
Conformal (preserve angles)
uniform scale

Rigid (preserve lengths)
rotate, translate

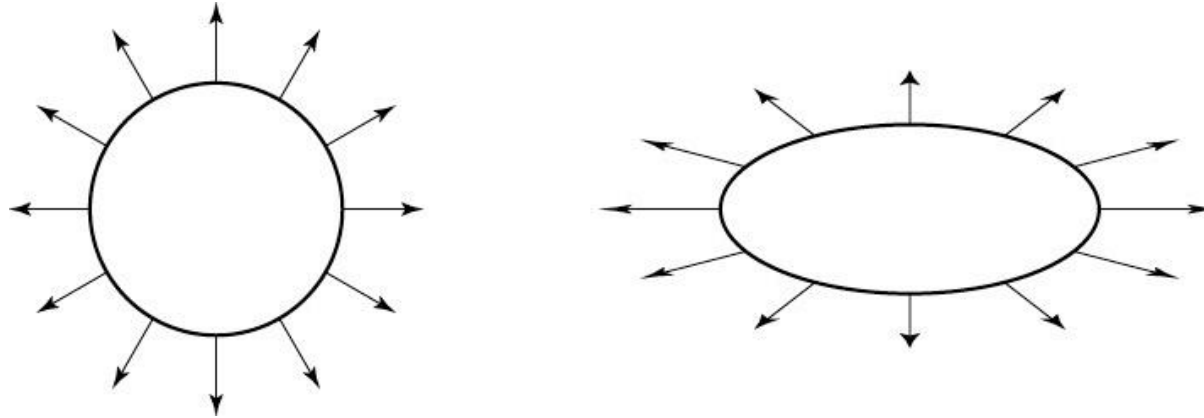


Composing Transformations

Any sequence of linear transforms can be concatenated into a single 3x3 matrix.
In general transforms DO NOT commute (some special combinations are).



Transforming tangents and normals



Tangents can be written as the difference of points on an object.
Say $\mathbf{t} = \mathbf{p} - \mathbf{q}$. The transformed tangent $\mathbf{t}' = \mathbf{M}\mathbf{p} - \mathbf{M}\mathbf{q} = \mathbf{M}(\mathbf{p} - \mathbf{q}) = \mathbf{M}\mathbf{t}$.

But $\mathbf{n}' \neq \mathbf{M}\mathbf{n}$ for surface normals. (imagine scaling an object in x-direction)
Say $\mathbf{n}' = \mathbf{H}\mathbf{n}$. What is $\mathbf{H}$?

$$\begin{aligned}\mathbf{t}'^T \mathbf{n}' &= 0 \\ (\mathbf{M}\mathbf{t})^T (\mathbf{H}\mathbf{n}) &= 0 \\ \mathbf{t}^T (\mathbf{M}^T \mathbf{H}) \mathbf{n} &= 0\end{aligned}$$

Remember that $\mathbf{t}^T \mathbf{n} = 0$. so... $\mathbf{H} = (\mathbf{M}^T)^{-1}$



Representing 3D transforms as a 4x4 matrix

Translate a point $[x \ y \ z]^T$ by $[t_x \ t_y \ t_z]^T$:

$$\begin{pmatrix} x' \\ y' \\ z' \\ 1 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix}$$

Rotate a point $[x \ y \ z]^T$ by an angle t **around z axis**:

$$\begin{pmatrix} x' \\ y' \\ z' \\ 1 \end{pmatrix} = \begin{pmatrix} \cos t & -\sin t & 0 & 0 \\ \sin t & \cos t & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix}$$

Scale a point $[x \ y \ z]^T$ by a factor $[s_x \ s_y \ s_z]^T$

$$\begin{pmatrix} x' \\ y' \\ z' \\ 1 \end{pmatrix} = \begin{pmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix}$$

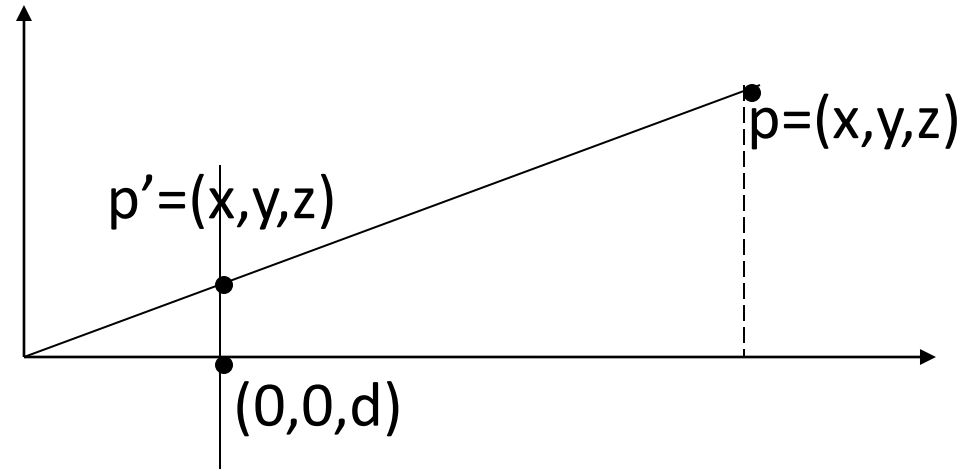


Perspective Projection

$$y' = yd/z$$

$$x' = xd/z$$

$$z' = d$$



Insight: $w' = z/d$

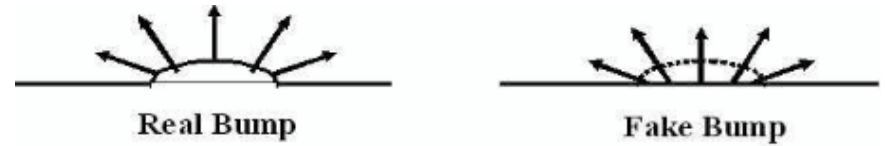
$$\begin{pmatrix} x' \\ y' \\ z' \\ w' \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 1/d & 0 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix}$$



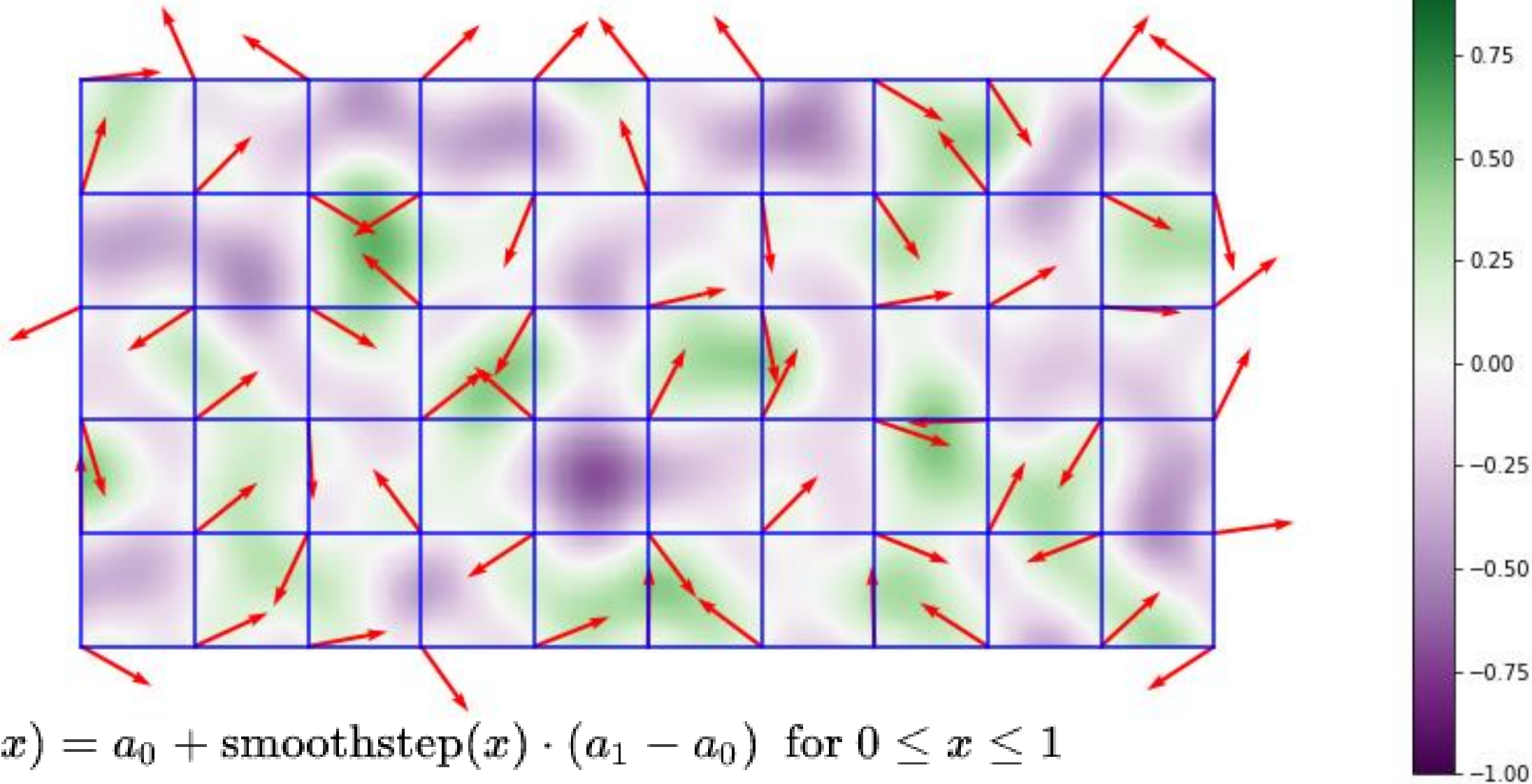
Bump | Normal Mapping

One of the reasons why we apply texture mapping:

Real surfaces are hardly flat but often rough and bumpy. These bumps cause (slightly) different reflections of the light.



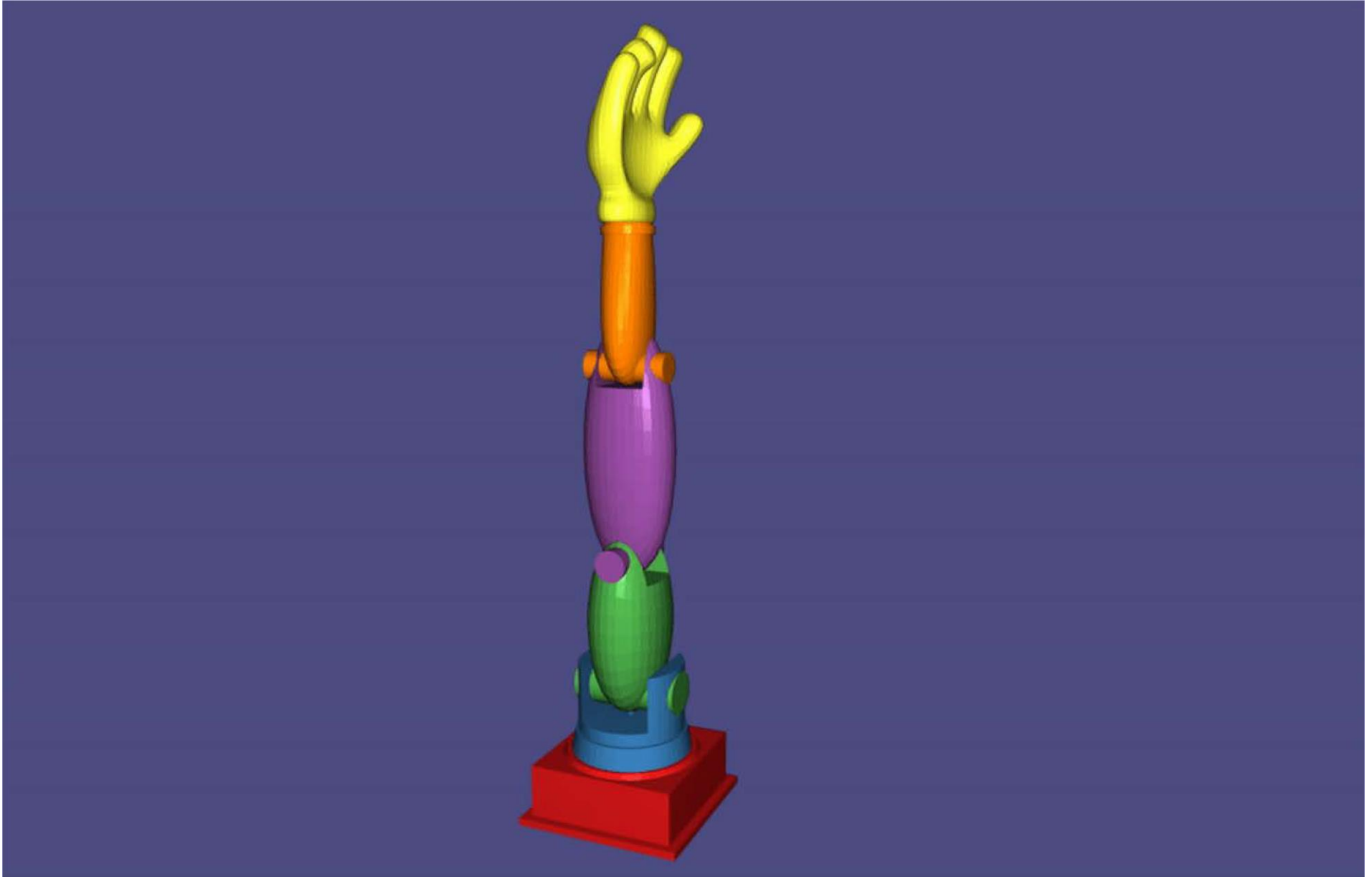
Perlin Noise



$$f(x) = a_0 + \text{smoothstep}(x) \cdot (a_1 - a_0) \quad \text{for } 0 \leq x \leq 1$$

$$\text{smoothstep}(x) = S_1(x) = \begin{cases} 0 & x \leq 0 \\ 3x^2 - 2x^3 & 0 \leq x \leq 1 \\ 1 & 1 \leq x \end{cases}$$

Kinematics



Animation Principles

Squash & Stretch

Timing

Anticipation

Staging

Follow-through & Overlapping
Action

Straight-Ahead vs. Pose-to-Pose

Ease-In & Ease-Out

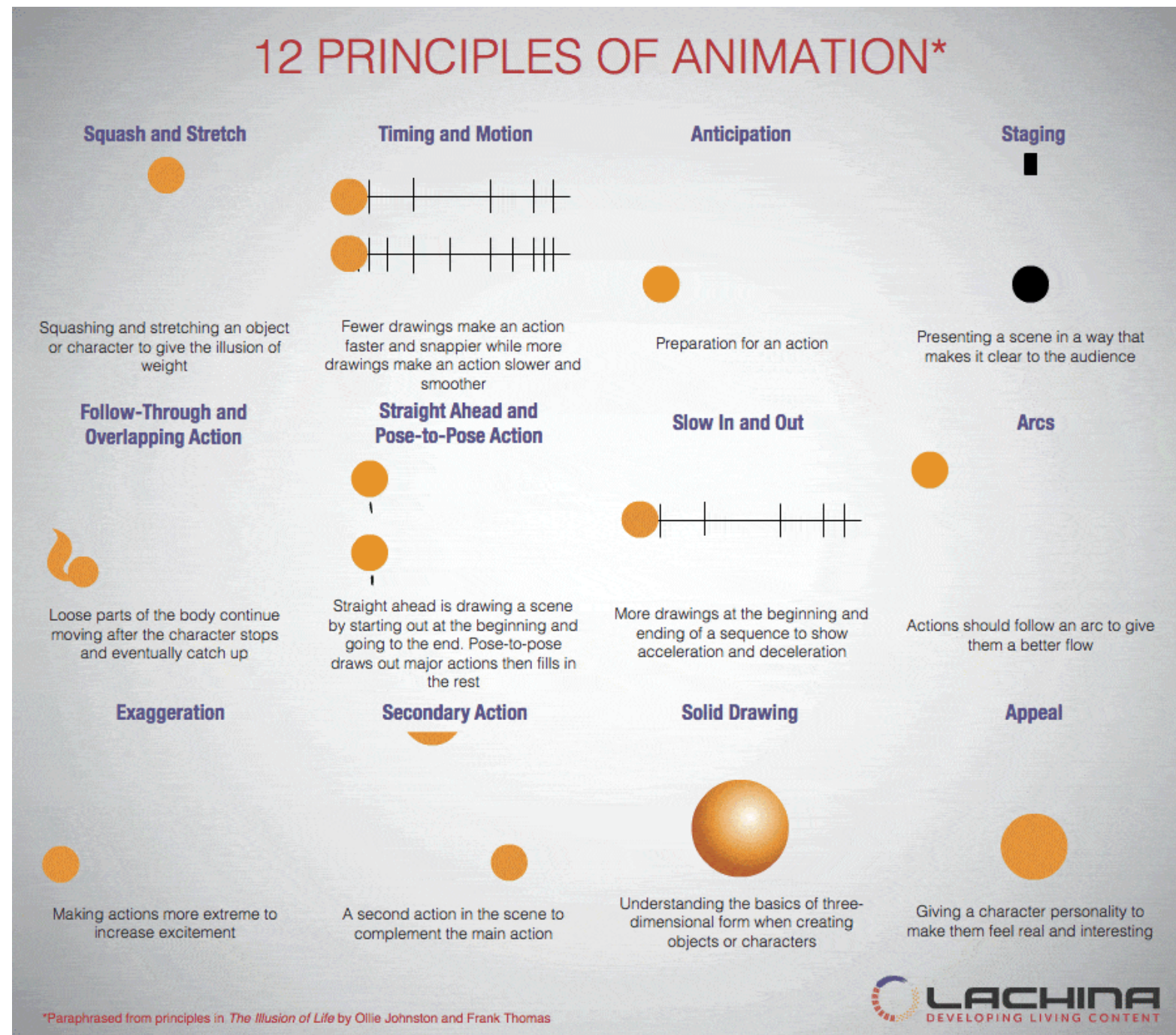
Arcs

Exaggeration

Secondary Motion

Solid Drawing

Appeal



What can be animated?

- Lights
- Camera
- **Jointed figures**
- **Skin/muscles**
- **Deformable objects**
- Clothing
- Wind/water/fire/smoke
- Hair

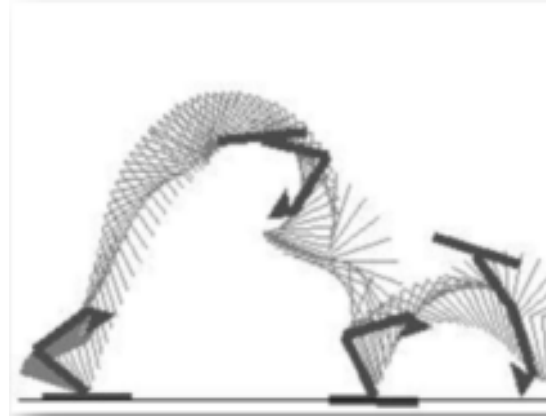
...any variable, Given the right time scale, almost anything...

Approaches to Animation

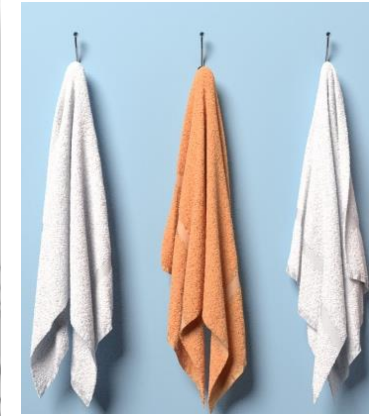
How does one make digital models move?



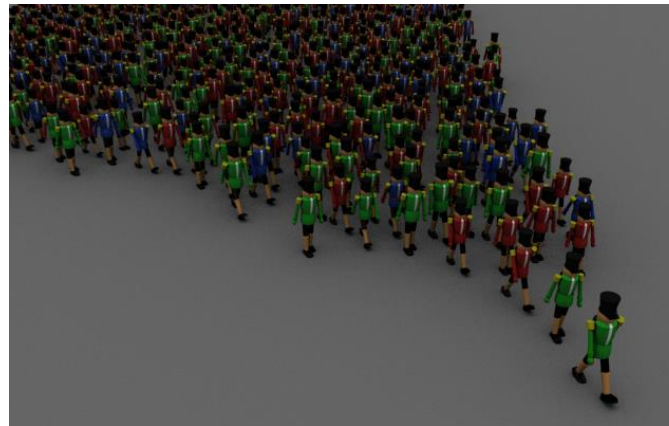
Keyframing



Physical simulation



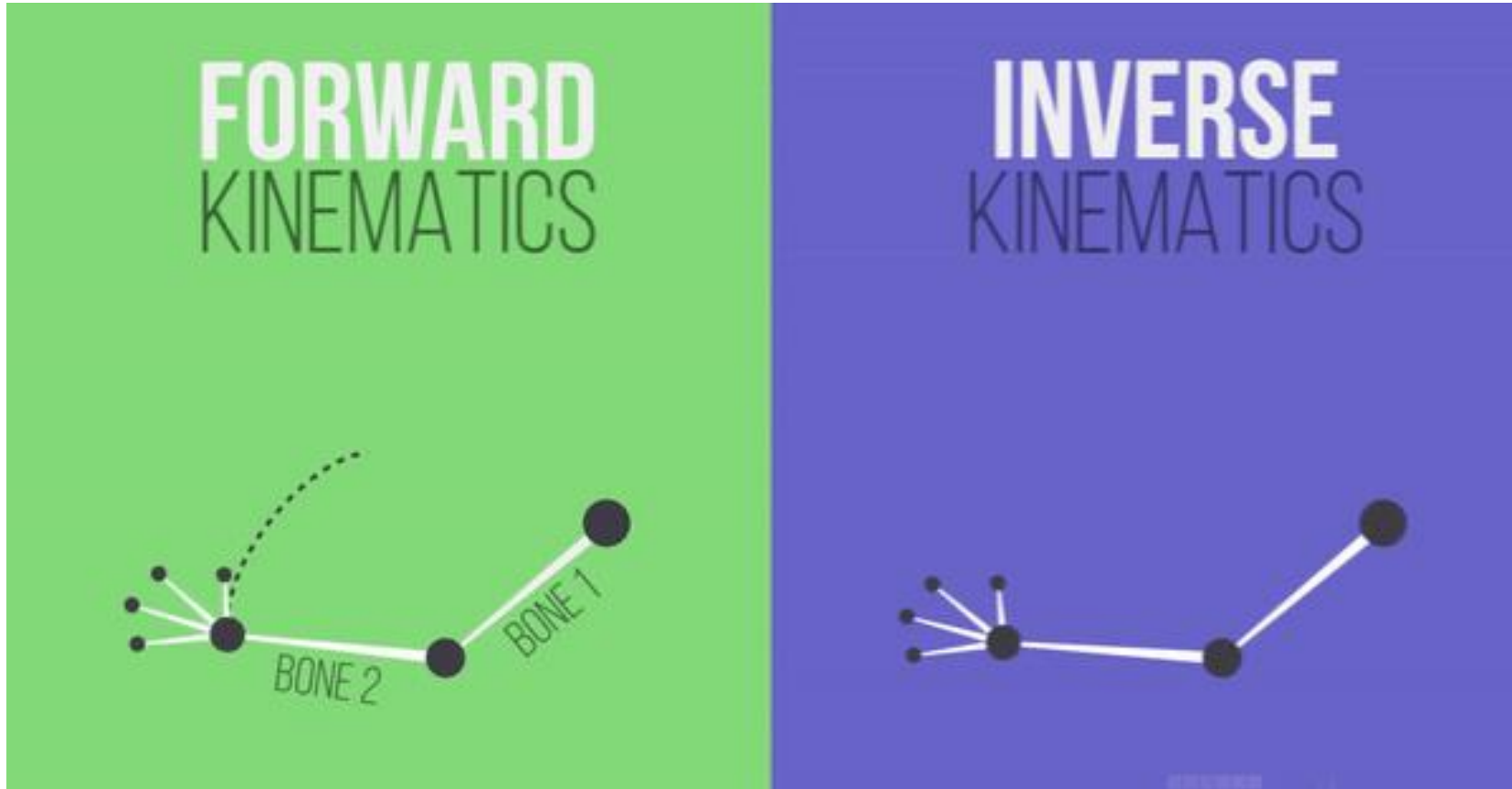
Motion capture



Behavior rules

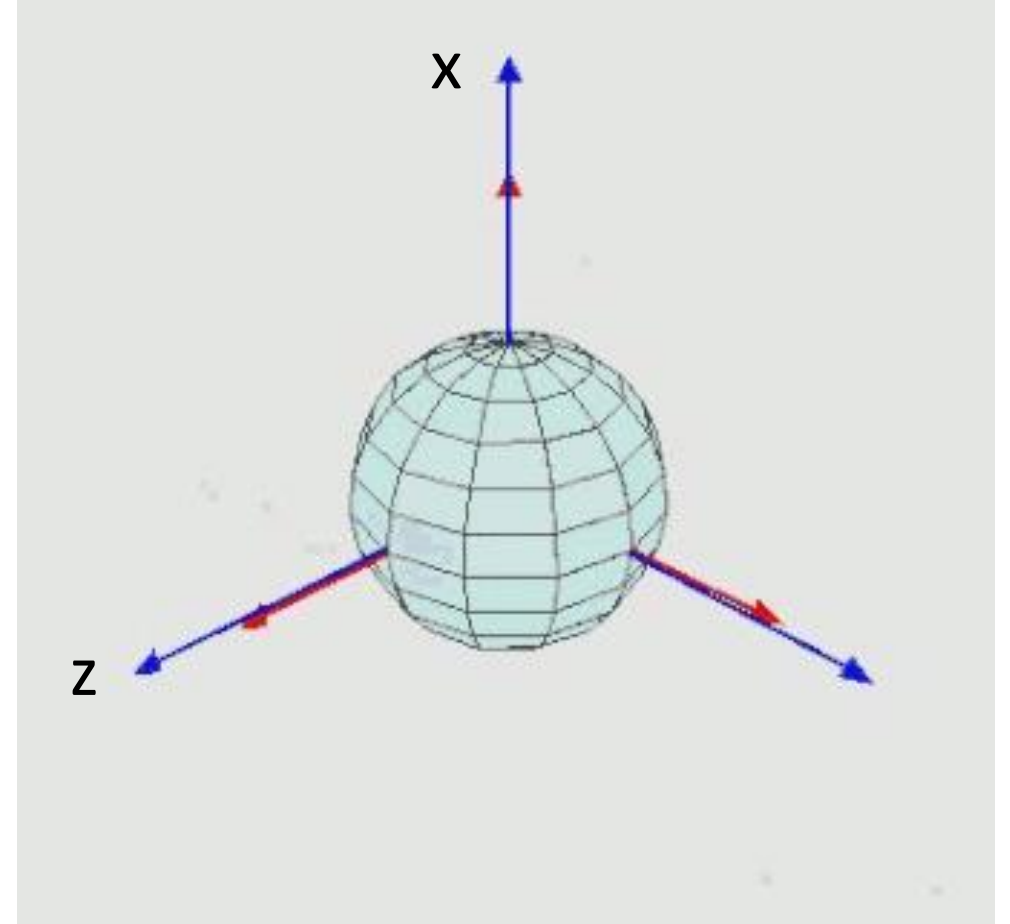
Posing a skeleton:

Forward Kinematics vs. Inverse Kinematics



Euler Angle Rotations

$$R_x(\theta_3) * R_z(\theta_2) * R_x(\theta_1)$$



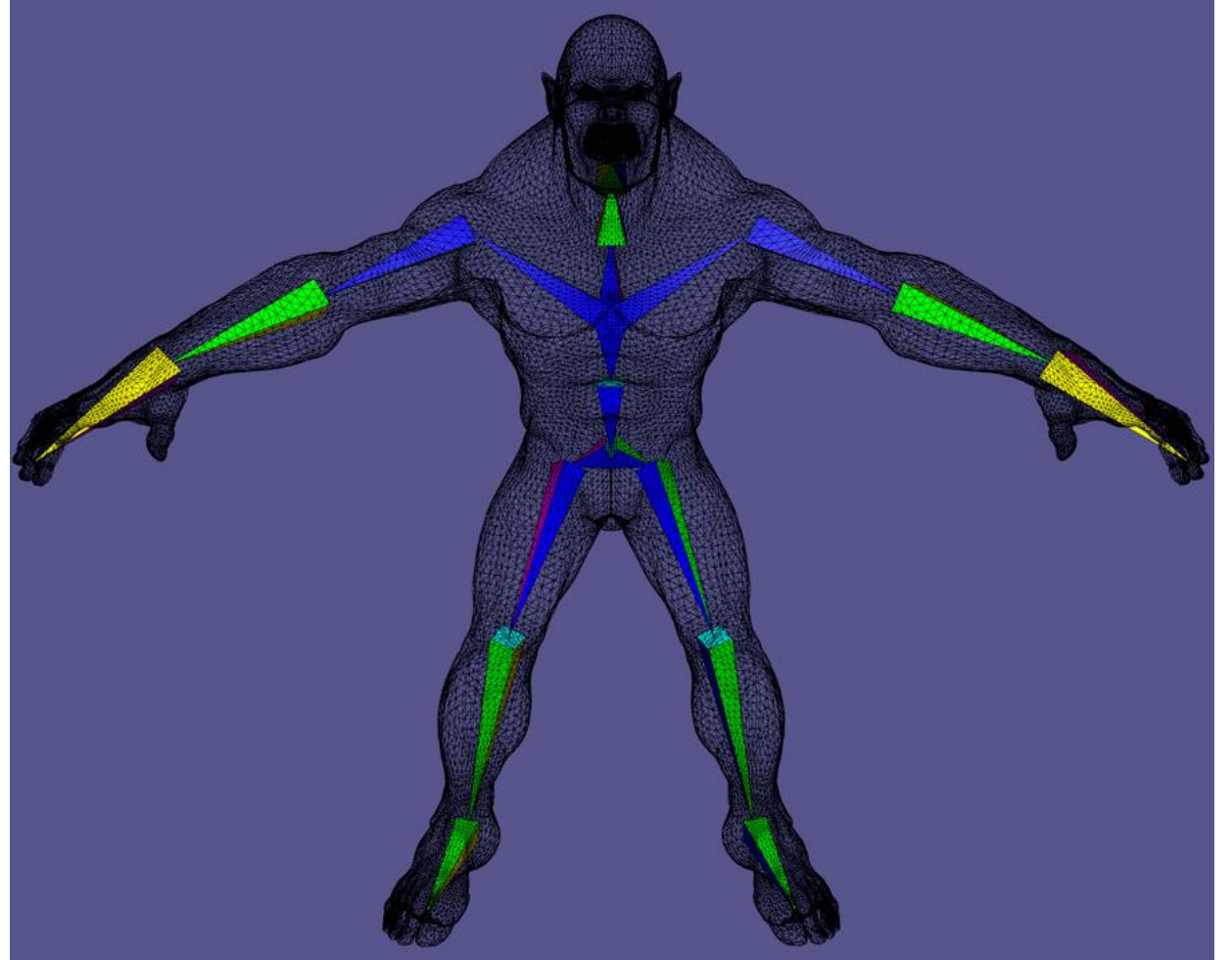
https://en.wikipedia.org/wiki/Euler_angles

<https://mathworld.wolfram.com/EulerAngles.html>

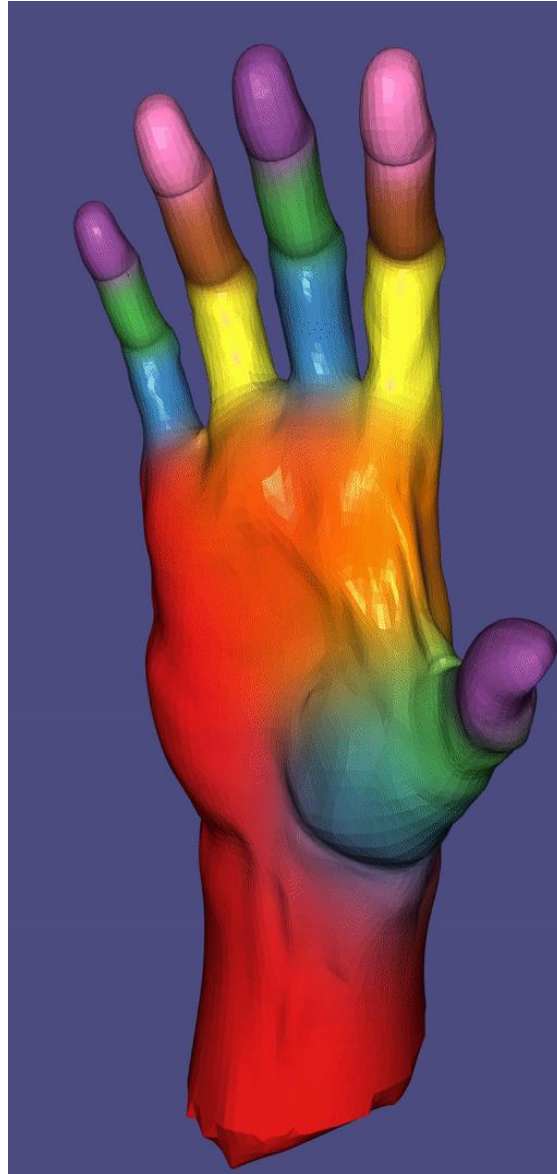
Skeletons: Forward Kinematics

$$\mathbf{T}_i = \mathbf{T}_{p_i} \begin{pmatrix} \hat{\mathbf{T}}_i & \\ & 0 \ 0 \ 0 \ 1 \end{pmatrix} \begin{pmatrix} \overline{\mathbf{R}}_i & 0 \\ & 0 \ 0 \ 0 \ 1 \end{pmatrix} \begin{pmatrix} \hat{\mathbf{T}}_i & \\ & 0 \ 0 \ 0 \ 1 \end{pmatrix}^{-1}$$

$$\mathbf{T}_i = \mathbf{T}_{p_i} \hat{\mathbf{T}}_i \begin{pmatrix} \mathbf{R}_x(\theta_{i3}) & 0 \\ & 0 \ 0 \ 0 \ 1 \end{pmatrix} \begin{pmatrix} \mathbf{R}_z(\theta_{i2}) & 0 \\ & 0 \ 0 \ 0 \ 1 \end{pmatrix} \begin{pmatrix} \mathbf{R}_x(\theta_{i1}) & 0 \\ & 0 \ 0 \ 0 \ 1 \end{pmatrix} \hat{\mathbf{T}}_i^{-1}$$



Linear Blend Skinning



$$\mathbf{v}_j = \sum_{i=1}^{\text{\#bones}} w_{ij} \mathbf{T}_i \hat{\mathbf{v}}_j$$

Specifying Keyframes



Time = 0



Time = 10s

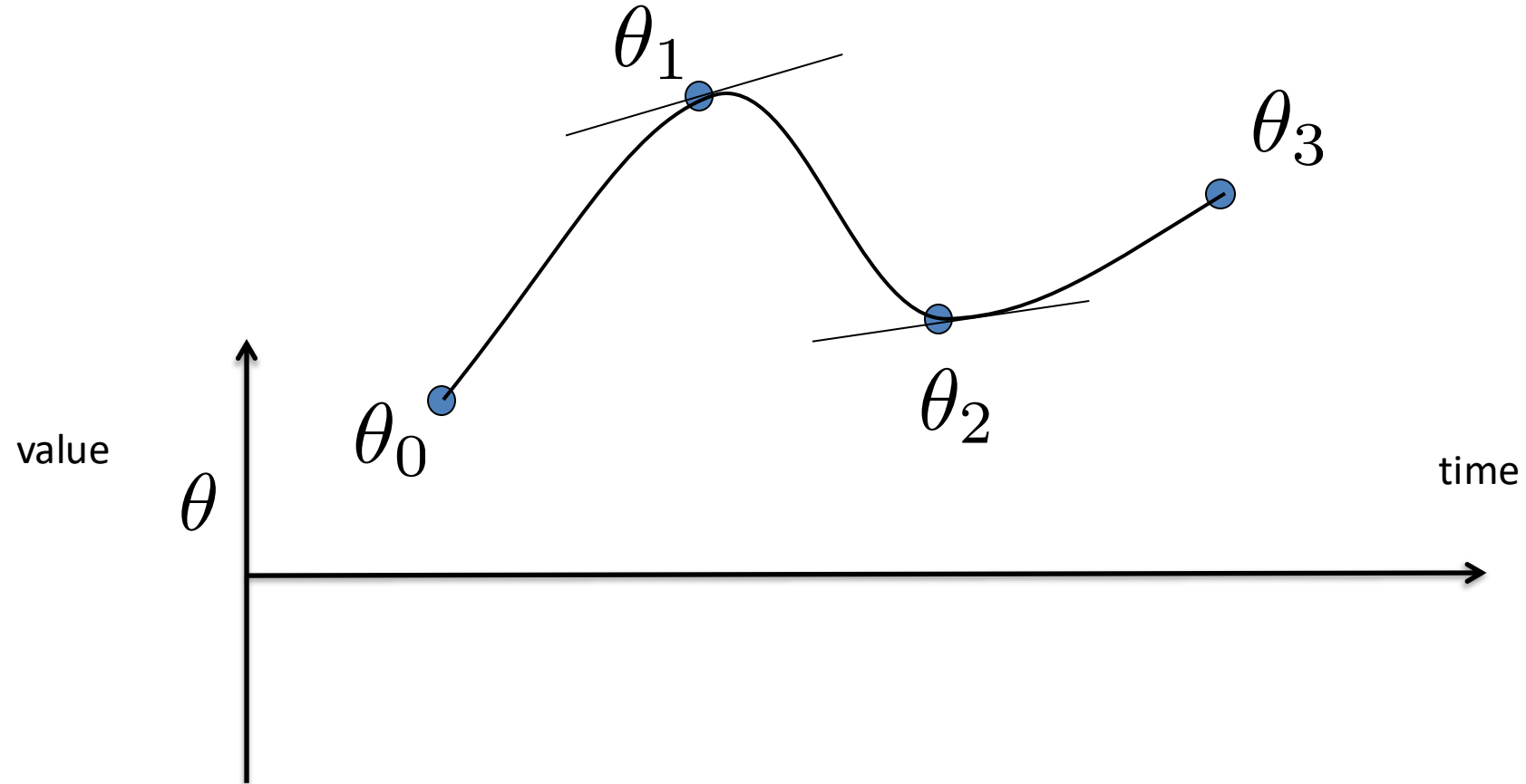
Poses are generated by specifying rotations of bones

Each pose can be represented as

$$\theta = \begin{pmatrix} \theta_{11} \\ \theta_{11} \\ \theta_{11} \\ \vdots \\ \theta_{n1} \\ \theta_{n2} \\ \theta_{n3} \end{pmatrix}$$

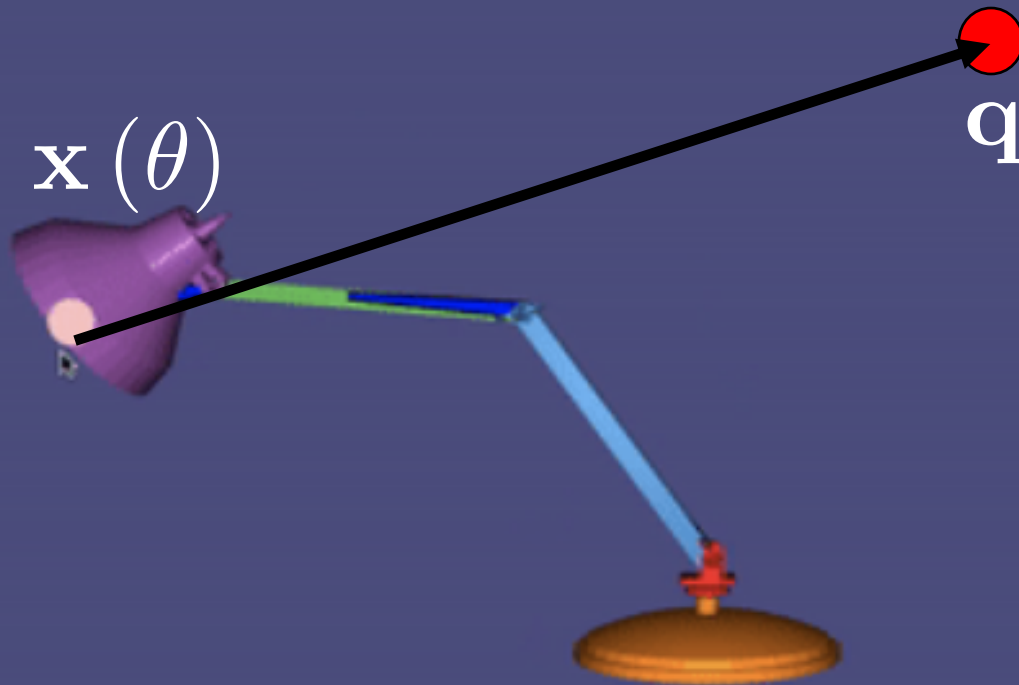
Catmull-Romm Spline

Catmull-Romm chooses the tangents using “Finite Differences”



$$\mathbf{m}_k = \frac{\theta_{k+1} - \theta_k}{t_{k+1} - t_k}$$

Inverse Kinematics



$$\theta^* = \arg \min \|\mathbf{x}(\theta) - \mathbf{q}\|_2^2$$

Skeletons: Inverse Kinematics

Closeness energy can be measured the squared distance between the pose tip $\mathbf{x}_b$ of some bone b and a desired goal location $\mathbf{q} \in \mathbb{R}^3$.

$$E(\mathbf{x}_b) = \|\mathbf{x}_b - \mathbf{q}\|^2.$$

Given pose vector $\mathbf{a}$, the bone tip $\mathbf{x}_b$ is:

$$\mathbf{x}_b(\mathbf{a}) = \mathbf{T}_b \hat{\mathbf{d}}_b$$

Now given any number of end-effectors $b_1, \dots, b_k$:

$$\min_{\mathbf{a}} \underbrace{\sum_{i=1}^k \|\mathbf{x}_{b_i}(\mathbf{a}) - \hat{\mathbf{x}}_{b_i}\|^2}_{E(\mathbf{x}_b(\mathbf{a}))}$$

And we impose some joint angle limits: $\min_{\mathbf{a}^{\min} \leq \mathbf{a} \leq \mathbf{a}^{\max}} E(\mathbf{x}_b(\mathbf{a}))$

Minimizing this energy is a non-linear least squares problem.



Skeletons: Inverse Kinematics Minimization

Projected Gradient Descent:

Start with an initial pose $\mathbf{a}$, and move in direction of decrease in E , project the pose to stay within limits and iterate towards solution.

$$\mathbf{a} \leftarrow \mathbf{a} - \sigma \left(\frac{dE(\mathbf{x}(\mathbf{a}))}{d\mathbf{a}} \right)^T$$

$$\mathbf{a} \leftarrow \mathbf{a} - \sigma \left(\frac{d\mathbf{x}(\mathbf{a})}{d\mathbf{a}} \right)^T \left(\frac{dE(\mathbf{x})}{d\mathbf{x}} \right) \quad \text{chain rule}$$

$$\frac{dE}{d\mathbf{a}} \in \mathbb{R}^{|\mathbf{a}|}, \quad \frac{dE}{d\mathbf{x}} \in \mathbb{R}^{|\mathbf{x}|}, \quad \text{and} \quad \frac{d\mathbf{x}}{d\mathbf{a}} \in \mathbb{R}^{|\mathbf{x}| \times |\mathbf{a}|}$$

$$\mathbf{J} = \frac{d\mathbf{x}}{d\mathbf{a}}. \quad \text{also known as Jacobian measures the change in } \mathbf{x} \text{ for changes in joint angles } \mathbf{a},$$

$$\mathbf{J} \text{ can be computed using Finite Differences:} \quad \mathbf{J}_{i,j} \approx \frac{\mathbf{x}_i(\mathbf{a} + h\delta_j)}{h}. \quad h = 10^{-7}$$

$$\left(\frac{dE(\mathbf{x})}{d\mathbf{x}} \right) \text{ is gradient of } \sum_{i=1}^k \|\mathbf{x}_{b_i}(\mathbf{a}) - \hat{\mathbf{x}}_{b_i}\|^2$$

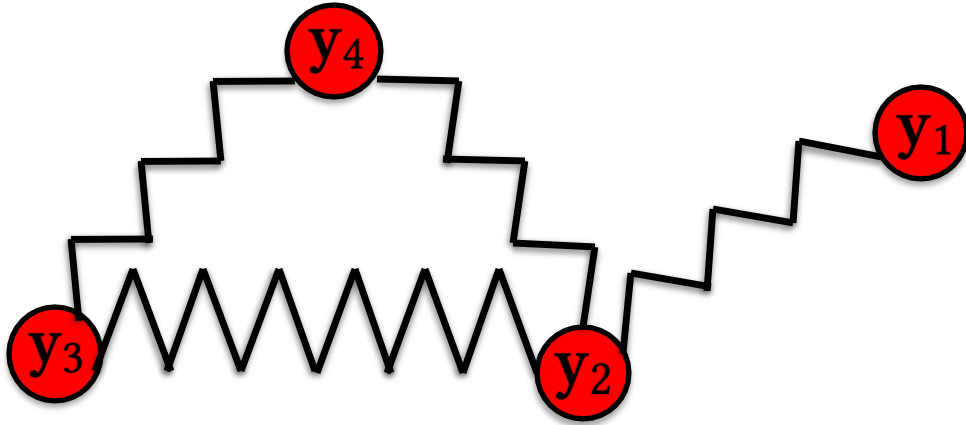
$$\text{Project to within limits: } \mathbf{a}_i \leftarrow \max[\mathbf{a}_i^{\min}, \min[\mathbf{a}_i^{\max}, \mathbf{a}_i]].$$

$$\text{Find a good step that lowers energy: } E(\text{proj}(\mathbf{a} + \sigma \Delta \mathbf{a})) < E(\mathbf{a}).$$

Mass-Springs



Newton's Second Law: System of Equations



$$F = ma = -kx$$

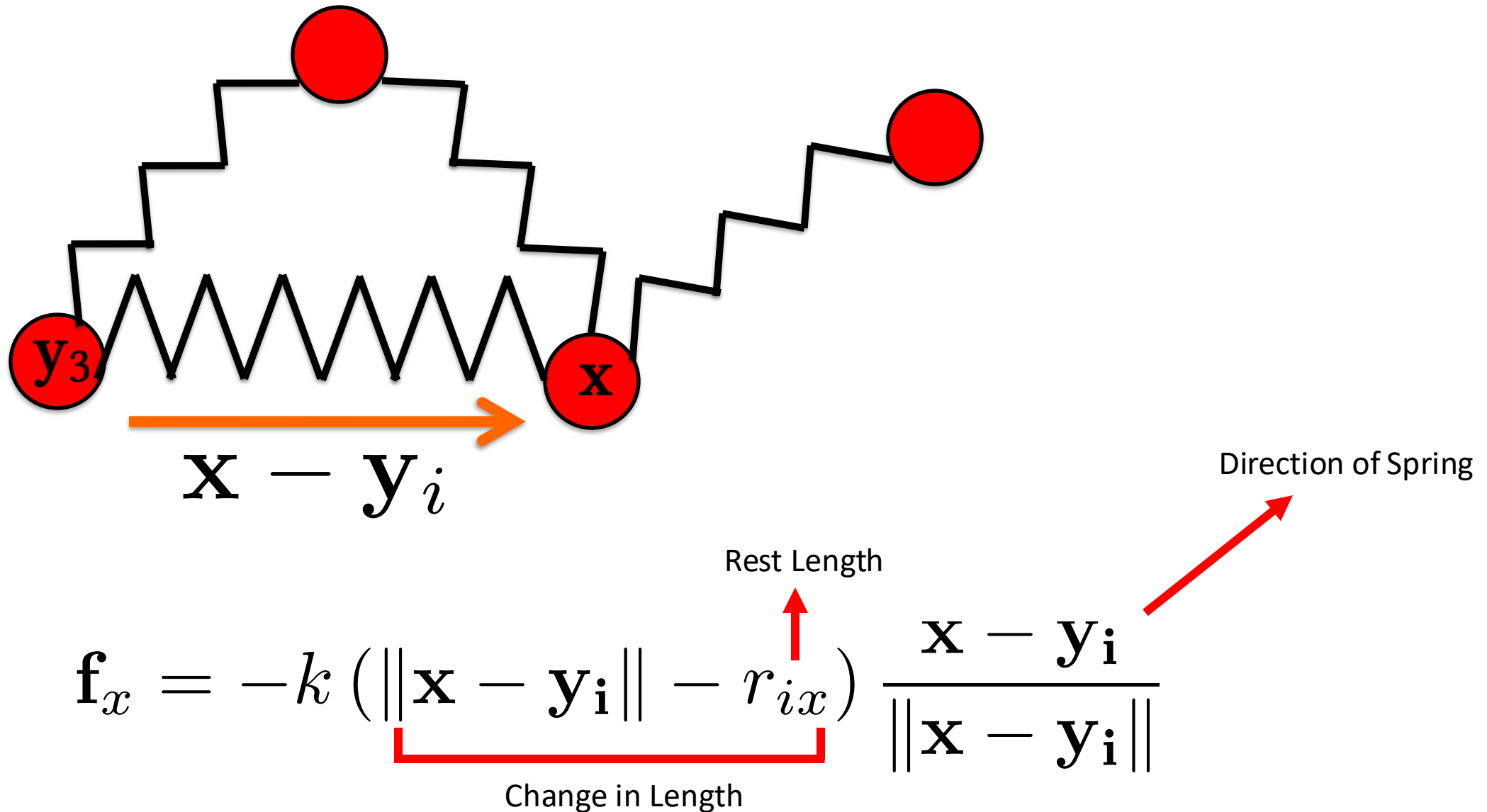
$$\begin{pmatrix} m_1 \cdot I & 0 & 0 & 0 \\ 0 & m_2 \cdot I & 0 & 0 \\ 0 & 0 & m_3 \cdot I & 0 \\ 0 & 0 & 0 & m_4 \cdot I \end{pmatrix} \begin{pmatrix} \mathbf{a}_1 \\ \mathbf{a}_2 \\ \mathbf{a}_3 \\ \mathbf{a}_4 \end{pmatrix} = \begin{pmatrix} \mathbf{f}_1 \\ \mathbf{f}_2 \\ \mathbf{f}_3 \\ \mathbf{f}_4 \end{pmatrix}$$

Mass Matrix

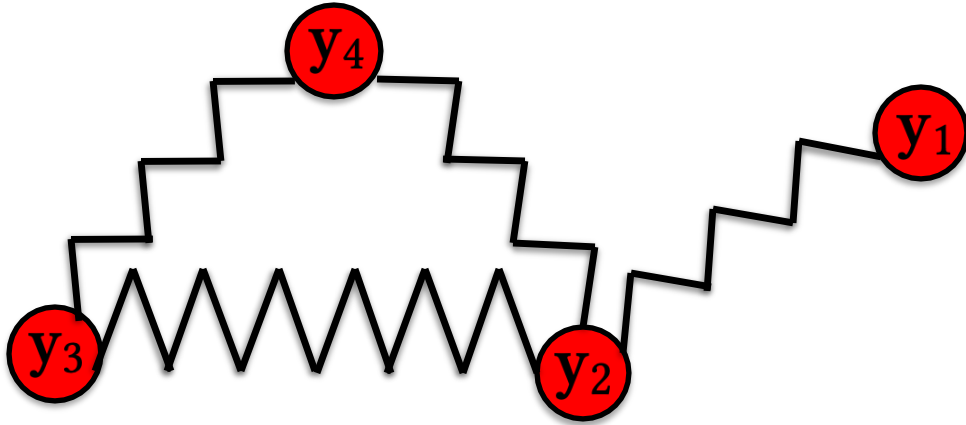
$\mathbf{a}(t)$ $\mathbf{f}(t)$



The Mass-Spring System



Time Integration



Need to Discretize!

$$M \frac{d^2 \mathbf{y}(t)}{dt^2} = \mathbf{f}(\mathbf{y}(t))$$

$$F = ma = \text{sprint_forces}$$

Use Finite Differences!

$$\frac{d^2\mathbf{y}(t)}{dt^2} \approx \frac{\mathbf{y}^{t+1} - 2\mathbf{y}^t + \mathbf{y}^{t-1}}{\Delta t^2}$$



Time Integration: Explicit vs. Implicit

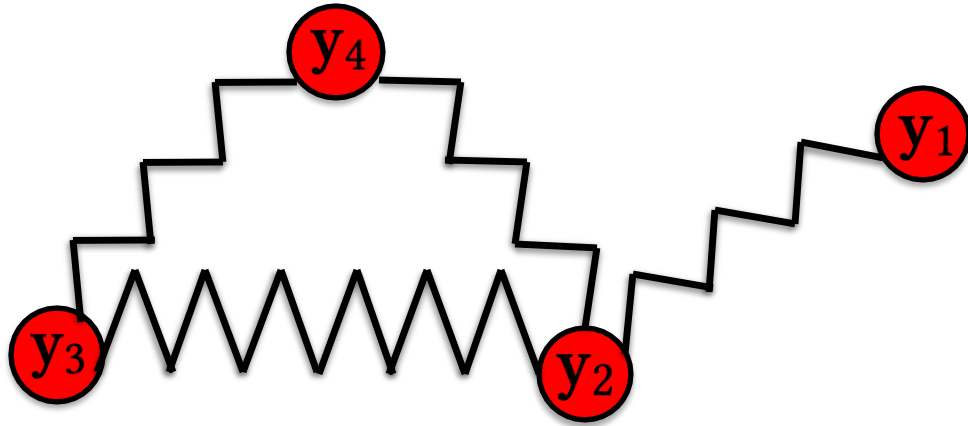
$$M \frac{d^2 \mathbf{y}(t)}{dt^2} = \mathbf{f}(\mathbf{y}(t))$$

Explicit: $\mathbf{y}_{t+dt} = g(\mathbf{y}_t)$. Future state $\mathbf{y}_{t+dt}$ is an explicit equation of current state $\mathbf{y}_t$ and dt .

Implicit: $h(\mathbf{y}_t, \mathbf{y}_{t+dt})=0$. Future state $\mathbf{y}_{t+dt}$ is an implicit equation.



Implicit Time Integration



$$M \left(\frac{\mathbf{y}^{t+1} - 2\mathbf{y}^t + \mathbf{y}^{t-1}}{\Delta t^2} \right) = \mathbf{f}(\mathbf{y}^{t+1})$$

$$M\mathbf{y}^{t+1} = M(2\mathbf{y}^t - \mathbf{y}^{t-1}) + \Delta t^2 \mathbf{f}(\mathbf{y}^{t+1})$$

Goal: Solve for $\mathbf{y}^{t+1}$



Implicit Integration as Optimization

Rather than directly solve:

$$M\mathbf{y}^{t+1} - M(2\mathbf{y}^t - \mathbf{y}^{t-1}) - \Delta t^2 \mathbf{f}(\mathbf{y}^{t+1}) = 0$$

spring forces +
external (e.g. gravity) forces

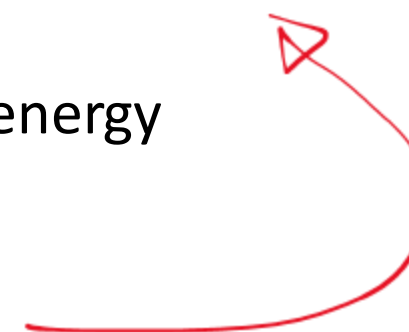


We can view the mass-force equations as the gradient of an energy function $\mathbf{E}(\mathbf{q})$ whose gradient vanishes as below:

$$\nabla_{\mathbf{q}} E(\mathbf{y}^{t+1}) = 0$$

Turning integration into an optimization problem:

$$\mathbf{y}^{t+1} = \arg \min_{\mathbf{q}} E(\mathbf{q})$$



Energy

$$\mathbf{y}^{t+1} = \underset{\mathbf{y}}{\operatorname{argmin}} \underbrace{\left(\sum_{ij} \frac{1}{2} k (\|\mathbf{y}_i - \mathbf{y}_j\| - r_{ij})^2 \right) - \frac{\Delta t^2}{2} \left(\sum_i m_i \left(\frac{\mathbf{y}_i - 2\mathbf{y}_i^t + \mathbf{y}_i^{t-1}}{\Delta t^2} \right)^2 \right)}_{E(\mathbf{y})} - \underbrace{\left(\sum_i \mathbf{y}_i^\top \mathbf{f}_i^{\text{ext}} \right)}_{\text{External forces}}$$

Potential energy force

$$V(\mathbf{y}_i, \mathbf{x}) = \frac{1}{2} k (\|\mathbf{y}_i - \mathbf{x}\| - r_{ix})^2$$

Spring energy

0.5*ma²
Kinetic energy-like

$$\mathbf{a}_i^t = \ddot{\mathbf{y}}_i^t = \frac{d^2 \mathbf{y}_i(t)}{dt^2} \approx \frac{\mathbf{y}_i^{t+1} - 2\mathbf{y}_i^t + \mathbf{y}_i^{t-1}}{\Delta t^2}$$



Observation!

$$(\|\mathbf{y}_i - \mathbf{y}_j\| - r_{ij})^2 = \min_{\mathbf{d}_{ij} \in \mathbb{R}^3, \|\mathbf{d}\|=r_{ij}} \|(\mathbf{y}_i - \mathbf{y}_j) - \mathbf{d}_{ij}\|^2$$

$$\mathbf{y}^{t+1} = \operatorname{argmin}_{\mathbf{y}} \underbrace{\left(\sum_{ij} \frac{1}{2} k \|\mathbf{y}_i - \mathbf{y}_j\| - \mathbf{d}_{ij} \|^2 \right)}_{E_2(\mathbf{y})} - \underbrace{\frac{\Delta t^2}{2} \left(\sum_i m_i \left(\frac{\mathbf{y}_i - 2\mathbf{y}_i^t + \mathbf{y}_i^{t-1}}{\Delta t^2} \right)^2 \right)}_{\tilde{E}(\mathbf{y})} - \underbrace{\left(\sum_i \mathbf{y}_i^\top \mathbf{f}_i^{\text{ext}} \right)}_{E_1(\mathbf{y})}$$

Make the terms linear

Quadratic!



Local-Global Solvers for Mass-Spring Systems

WHILE Not done

//Local Steps

For Each Spring

$$E_{ij} = \arg \min_{\mathbf{d}_{ij}, |\mathbf{d}_{ij}|=r_{ij}} \frac{k}{2} \|\mathbf{y}_i - \mathbf{y}_j - \mathbf{d}_{ij}\|^2$$

//Global Step

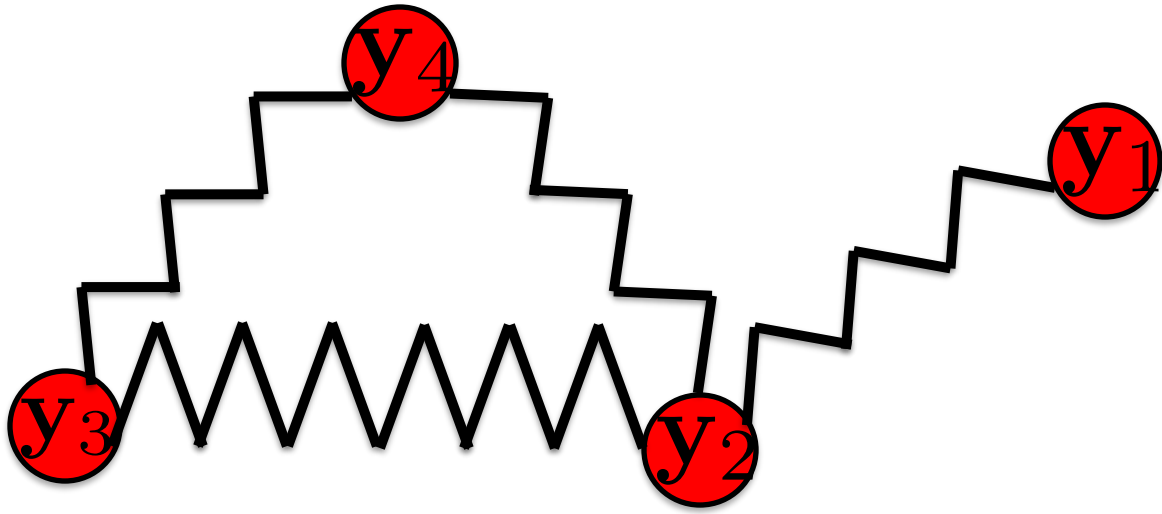
$$\text{Solve } (M + \Delta t^2 k G^T G) \mathbf{y} = (M \mathbf{b} + \Delta t^2 k \mathbf{G}^T \mathbf{d})$$

END

$$A\mathbf{y} = \mathbf{b}$$



Fixed Points via Projection



$$(M + \Delta t^2 k G^T G) \mathbf{y} = (M \mathbf{b} - \Delta t^2 k \mathbf{G}^T \mathbf{d})$$

Substituting $\mathbf{y} = P\tilde{\mathbf{y}} + \mathbf{c}$ in $A\mathbf{y} = \mathbf{f}$

Too many rows now ... $AP\tilde{\mathbf{y}} = \mathbf{f} - A\mathbf{c}$

$$P^T AP\tilde{\mathbf{y}} = P^T (\mathbf{f} - A\mathbf{c}) \quad \text{...Rebuild } \mathbf{y}$$



**This is the processing behind the pixels,
the math behind the magic!**

